

```

using System.IO;
using UnityEditor;
using UnityEditor.Animations;
using UnityEngine;
using UnityEngine.AI;
/// <summary>
/// 菜单: Tools/TPS/Build Animal Prefabs
/// 把 Art/Animals 美术包里的源 prefab 包成 Resources/Animals/*.prefab, 让运行时 Resources.Load 能加载。
/// 源 prefab 自带 Animator (控制器默认状态即 Idle), 但还挂着 NavMeshAgent / CharacterController /
/// AudioSource 以及美术包自带、未随美术导入的 demo AI 脚本 (在工程里是 missing script)。
/// 当前只要「站立 idle、无复杂行为」, 所以生成时会剥掉这些组件, 只保留模型层级 + Animator。
/// 复杂行为后续再在这些 prefab 上加 (预留)。
///
/// 死亡动画各动物的 Animator bool 参数名不一样(isDead / isDead_0 ...), 这里从 AnimatorController
/// 自动探测(取名字含 dead/die/death 的 bool 参数), 烘焙进 prefab 上的 AnimalEntity, 探测不到则回退 isDead。
///
/// 重跑会覆盖已有 prefab (guid 保留)。Art/Animals 下的源 prefab 不动。
/// 新增动物: 在 <see cref="SourceFiles"/> 加一行文件名, 并在 animal 配表 prefab 列填 Animals/{文件名}。
/// </summary>
public static class AnimalPrefabBuilder
{
    private const string ResourceFolder = "Assets/Resources";
    private const string OutSubfolder = "Animals";
    private const string PackRoot = "Assets/Art/Animals/Low Poly Animated Animals/Prefabs/Animals";
    // 与 animal 配表 prefab 列一一对应(Animals/{文件名})。野鸭->Goose(包内无鸭,用最接近的水禽),野兔/野猪复用为
    兔子/猪。
    private static readonly string[] SourceFiles =
    {
        // 原有 3 种
        "Goose", "Rabbit_Brown", "Boar",
        // 新增 19 种(熊/牛/骆驼/奶牛/鳄鱼/鹿/大象/长颈鹿/山羊/鸡/河马/马/狮子/麋鹿/犀牛/绵羊/老虎/狼/斑马)
        "Bear_Grizzly", "Cow_Horns_Brown", "Camel_Dromedary", "Cow_White", "Crocodile", "Deer",
        "Elephant_Male", "Giraffe", "Goat", "Hen", "Hippo", "Horse_Brown", "Lion_Male", "Reindeer",
        "Rhino", "Sheep_Wool", "Tiger", "Wolf", "Zebra",
    };
    [MenuItem("Tools/TPS/Build Animal Prefabs")]
    public static void Build()
    {
        EnsureFolder(ResourceFolder, "Assets", "Resources");
        EnsureFolder($"{ResourceFolder}/{OutSubfolder}", ResourceFolder, OutSubfolder);
        int ok = 0, fail = 0;
        foreach (var file in SourceFiles)
        {
            var srcPath = $"{PackRoot}/{file}.prefab";
            var src = AssetDatabase.LoadAssetAtPath<GameObject>(srcPath);
            if (src == null)
            {
                Debug.LogError($"[AnimalPrefab] 找不到源 prefab: {srcPath}");
                fail++;
                continue;
            }

```

```

}
var name = file;
var prefabPath = $"{ResourceFolder}/{OutSubfolder}/{name}.prefab";
// 实例化后彻底解包成独立层级（不再链接美术源 prefab），再剥掉非 idle 组件
var instance = (GameObject)PrefabUtility.InstantiatePrefab(src);
try
{
    PrefabUtility.UnpackPrefabInstance(instance, PrefabUnpackMode.Completely, InteractionMode.Autom
atedAction);
    StripToldle(instance);
    AddHitZones(instance);
    var animator = instance.GetComponentInChildren<Animator>(true);
    if (animator == null || animator.runtimeAnimatorController == null)
        Debug.LogWarning($"[AnimalPrefab] {name} 没有 Animator 或未挂控制器，idle 可能不会播放");
    // 烘焙 AnimalEntity 并自动探测死亡参数名(运行时 AnimalSystem 只覆盖 animalId/score,保留这里的
deathBool)
    var entity = instance.GetComponent<AnimalEntity>();
    if (entity == null) entity = instance.AddComponent<AnimalEntity>();
    entity.deathBool = DetectDeathBool(animator, name);
    PrefabUtility.SaveAsPrefabAsset(instance, prefabPath, out bool success);
    if (success)
    {
        Debug.Log($"[AnimalPrefab] 生成 {prefabPath}");
        ok++;
    }
    else
    {
        Debug.LogError($"[AnimalPrefab] 保存失败: {prefabPath}");
        fail++;
    }
}
finally
{
    Object.DestroyImmediate(instance);
}
}
AssetDatabase.SaveAssets();
AssetDatabase.Refresh();
Debug.Log($"[AnimalPrefab] 完成: 成功 {ok}, 失败 {fail}");
}
/// <summary>
/// 从动物的 AnimatorController 自动探测死亡动画对应的 bool 参数名:取第一个名字含 dead/die/death 的 bool 参
数。
/// 探测不到(无控制器/无此类参数)则回退 "isDead",此时运行时死亡动画可能不播,但仍会按时销毁(见 AnimalEntity)
。
/// </summary>
private static string DetectDeathBool(Animator animator, string name)
{
    var rac = animator != null ? animator.runtimeAnimatorController : null;
    // 源 prefab 一般直接挂 AnimatorController;若是 Override,取其底层控制器

```

```

var ac = rac as AnimatorController;
if (ac == null && rac is AnimatorOverrideController over) ac = over.runtimeAnimatorController as AnimatorController;
if (ac == null) return "isDead";
foreach (var p in ac.parameters)
{
    if (p.type != AnimatorControllerParameterType.Bool) continue;
    var n = p.name.ToLowerInvariant();
    if (n.Contains("dead") || n.Contains("die") || n.Contains("death"))
        return p.name;
}
Debug.LogWarning($"[AnimalPrefab] {name} 未在 Animator 找到死亡 bool 参数,回退 isDead(死亡动画可能不播)");
return "isDead";
}
/// <summary>剥掉一切会驱动复杂行为/移动的组件, 只留模型 + Animator (控制器默认状态即 idle) 。</summary>
private static void StripToldle(GameObject root)
{
    foreach (var t in root.GetComponentsInChildren<Transform>(true))
    {
        var go = t.gameObject;
        DestroyAll<NavMeshAgent>(go);
        DestroyAll<CharacterController>(go);
        DestroyAll<AudioSource>(go);
        // 美术包自带的 demo AI 脚本未导入工程, 是 missing script, 一并清掉
        GameObjectUtility.RemoveMonoBehavioursWithMissingScript(go);
    }
}
/// <summary>
/// 按模型渲染包围盒切出三个不重叠的部位碰撞体(各挂 <see cref="AnimalHitZone"/>):
/// 头 = 体长方向「前 40%」的上半;
/// 心脏 = 「前 40%」的下半(前胸);
/// 身体 = 余下「后 60%」全高。
/// 三块拼起来 ≈ 整只动物,射线打中哪块就算哪个部位(头/心脏一枪死,身体两枪)。
/// 「前」是体长轴(取 X/Z 中较长者)的哪一端,优先按名字含 head 的骨骼判定,找不到默认 + 轴。
/// spawn 时根节点缩放会一并作用到碰撞体上。
/// </summary>
private static void AddHitZones(GameObject root)
{
    // 统一用我们切的三块:先清掉源模型可能自带的碰撞体
    foreach (var c in root.GetComponentsInChildren<Collider>(true)) Object.DestroyImmediate(c, true);
    var renderers = root.GetComponentsInChildren<Renderer>(true);
    if (renderers.Length == 0) return;
    // 实例化时根节点在原点且为单位变换,世界包围盒可直接当局部盒用
    var b = renderers[0].bounds;
    for (int i = 1; i < renderers.Length; i++) b.Encapsulate(renderers[i].bounds);
    Vector3 min = b.min, max = b.max, size = b.size, center = b.center;
    bool axisZ = size.z >= size.x; // 较长的水平轴 = 体长轴
    float len = axisZ ? size.z : size.x;
    float front = 0.40f * len; // 前 40% 为头+心脏区

```

```

float midY = center.y;           // 中线分上(头)/下(心脏)
// 判定「前」在体长轴哪一端
float headCoord = HeadCoordOnAxis(root, axisZ, out bool found);
float centerCoord = axisZ ? center.z : center.x;
bool frontPositive = !found || headCoord >= centerCoord;
float aMin = axisZ ? min.z : min.x;
float aMax = axisZ ? max.z : max.x;
float frontLo, frontHi, rearLo, rearHi;
if (frontPositive) { frontHi = aMax; frontLo = aMax - front; rearLo = aMin; rearHi = aMax - front; }
else { frontLo = aMin; frontHi = aMin + front; rearLo = aMin + front; rearHi = aMax; }
if (axisZ)
{
    MakeZoneBox(root, "Hit_Head", min.x, max.x, midY, max.y, frontLo, frontHi, HitZone.Head);
    MakeZoneBox(root, "Hit_Heart", min.x, max.x, min.y, midY, frontLo, frontHi, HitZone.Heart);
    MakeZoneBox(root, "Hit_Body", min.x, max.x, min.y, max.y, rearLo, rearHi, HitZone.Body);
}
else
{
    MakeZoneBox(root, "Hit_Head", frontLo, frontHi, midY, max.y, min.z, max.z, HitZone.Head);
    MakeZoneBox(root, "Hit_Heart", frontLo, frontHi, min.y, midY, min.z, max.z, HitZone.Heart);
    MakeZoneBox(root, "Hit_Body", rearLo, rearHi, min.y, max.y, min.z, max.z, HitZone.Body);
}
}
/// <summary>给某部位盒加一层贴合的发光高亮盒(头/心脏/身体各一色)。无碰撞、不投影,只做视觉标注。</summary>
private static void AddGlow(GameObject zoneBox, Vector3 size, HitZone zone)
{
    var glow = GameObject.CreatePrimitive(PrimitiveType.Cube);
    glow.name = "Glow";
    var col = glow.GetComponent<Collider>();
    if (col != null) Object.DestroyImmediate(col); // 高亮盒不参与命中判定
    glow.transform.SetParent(zoneBox.transform, false);
    glow.transform.localPosition = Vector3.zero;
    glow.transform.localRotation = Quaternion.identity;
    glow.transform.localScale = size; // 内置 Cube 是 1×1×1,缩放后正好贴合部位盒
    var mr = glow.GetComponent<MeshRenderer>();
    mr.sharedMaterial = GetGlowMaterial(zone);
    mr.shadowCastingMode = UnityEngine.Rendering.ShadowCastingMode.Off;
    mr.receiveShadows = false;
}
/// <summary>取(没有则建)部位高亮材质:头=暖黄、心脏=红、身体=绿,均用 Hunting/WeakPointHighlight。</summary>
private static Material GetGlowMaterial(HitZone zone)
{
    const string dir = "Assets/Art/Effects";
    string name; Color color;
    switch (zone)
    {
        case HitZone.Head: name = "WeakHead"; color = new Color(1f, 0.85f, 0.2f); break; // 暖黄
        case HitZone.Heart: name = "WeakHeart"; color = new Color(1f, 0.22f, 0.16f); break; // 红
        default: name = "WeakBody"; color = new Color(0.25f, 1f, 0.35f); break; // 绿
    }
}

```

```

string path = $"{dir}/{name}.mat";
var existing = AssetDatabase.LoadAssetAtPath<Material>(path);
if (existing != null) return existing;
var shader = Shader.Find("Hunting/WeakPointHighlight");
if (shader == null)
{
    Debug.LogError("[AnimalPrefab] 找不到 Hunting/WeakPointHighlight shader,部位高亮未生成");
    return null;
}
if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
var mat = new Material(shader) { name = name };
mat.SetColor("_Color", color);
AssetDatabase.CreateAsset(mat, path);
return mat;
}

/// <summary>名字含 "head" 的骨骼在体长轴上的世界坐标(用于判定动物朝向);找不到 found=false。</summary>
private static float HeadCoordOnAxis(GameObject root, bool axisZ, out bool found)
{
    foreach (var t in root.GetComponentsInChildren<Transform>(true))
    {
        if (t.name.ToLowerInvariant().Contains("head"))
        {
            found = true;
            return axisZ ? t.position.z : t.position.x;
        }
    }
    found = false;
    return 0f;
}

/// <summary>按世界轴对齐区间建一个部位碰撞体子物体(root 在原点单位变换,世界=局部)。</summary>
private static void MakeZoneBox(GameObject root, string name,
    float xMin, float xMax, float yMin, float yMax, float zMin, float zMax, HitZone zone)
{
    var go = new GameObject(name, typeof(BoxCollider), typeof(AnimalHitZone));
    go.transform.SetParent(root.transform, true);
    go.transform.position = new Vector3((xMin + xMax) * 0.5f, (yMin + yMax) * 0.5f, (zMin + zMax) * 0.5f);
    go.transform.localRotation = Quaternion.identity;
    var col = go.GetComponent<BoxCollider>();
    col.center = Vector3.zero;
    var sizeV = new Vector3(Mathf.Abs(xMax - xMin), Mathf.Abs(yMax - yMin), Mathf.Abs(zMax - zMin));
    col.size = sizeV;
    go.GetComponent<AnimalHitZone>().zone = zone;
    // 每个部位叠一层发光高亮盒标注命中范围:头=黄、心脏=红、身体=绿
    AddGlow(go, sizeV, zone);
}

private static void DestroyAll<T>(GameObject go) where T : Component
{
    foreach (var c in go.GetComponents<T>())
        if (c != null) Object.DestroyImmediate(c, true);
}

```

```

private static void EnsureFolder(string fullPath, string parent, string leaf)
{
    if (AssetDatabase.IsValidFolder(fullPath)) return;
    AssetDatabase.CreateFolder(parent, leaf);
}
}

#if UNITY_EDITOR
using System.Collections.Generic;
using System.IO;
using UnityEditor;
using UnityEngine;

/// <summary>
/// 菜单: Tools/TPS/Build Chapter1 Desert Map
/// 用 Nature 美术包里的自然景观预制体(裂地/仙人掌/枯树/猴面包树/石头/头骨/云)程序化拼出
/// 第一章「沙漠」场景预制体 <c>Resources/Map/Chapter1_Desert.prefab</c>。
///
/// 结构:
/// Chapter1_Desert
///   └─ Ground      ── 一整块沙色平面(URP 材质,自带 MeshCollider,顶面 y=0)+ 压扁的裂地斑块做地表细节
///   └─ Props        ── 仙人掌/枯树/猴面包树/石头/头骨,散布在中心活动区(半径 PlayRadius)之外做边框点缀
///   └─ Clouds       ── 高空云朵
///
/// 装饰物会被剥掉碰撞体(只保留 Ground 的地面碰撞体),避免挡住射击射线;一切按固定随机种子摆放,可复现。
/// 运行时由 <see cref="GameStartScene"/> 按选中关卡的 map 配表 scenePath 列实例化(沙漠=Map/Chapter1_Desert)。
///
/// 重跑覆盖旧 prefab。换/加装饰:改 <see cref="Scatter"/> 里的清单即可。
/// </summary>
public static class Chapter1DesertMapBuilder
{
    private const string NaturePack = "Assets/Art/Animals/Low Poly Animated Animals/Prefabs/Nature";
    private const string OutDir = "Assets/Resources/Map";
    private const string OutPath = OutDir + "/Chapter1_Desert.prefab";
    private const float GroundSize = 140f; // 地面边长(米);相机远眺也铺满
    private const float PlayRadius = 16f; // 中心活动区半径:动物/玩家在此,大装饰物避开
    private const int Seed = 20240601; // 固定种子,保证每次生成一致
    /// <summary>一类散布装饰:预制体名 + 数量 + 缩放区间。</summary>
    private readonly struct Deco
    {
        public readonly string Name;
        public readonly int Count;
        public readonly float MinScale;
        public readonly float MaxScale;
        public Deco(string name, int count, float minScale, float maxScale)
        {
            Name = name; Count = count; MinScale = minScale; MaxScale = maxScale;
        }
    }

    private static readonly Deco[] Scatter =
    {
        new Deco("Cactus_Big", 16, 0.8f, 1.4f),
    }

```

```

    new Deco("Tree_Dead", 10, 0.9f, 1.5f),
    new Deco("Tree_Baobab", 5, 0.9f, 1.3f),
    new Deco("Stone_Flat", 14, 0.8f, 1.8f),
    new Deco("Skull_Human", 6, 0.8f, 1.2f),
};
private static readonly Deco[] Clouds =
{
    new Deco("Cloud_Big", 5, 1.0f, 1.8f),
    new Deco("Cloud_Long", 4, 1.0f, 1.6f),
};
[MenuItem("Tools/TPS/Build Chapter1 Desert Map")]
public static void Build()
{
    if (!Directory.Exists(OutDir)) Directory.CreateDirectory(OutDir);
    Random.InitState(Seed);
    var root = new GameObject("Chapter1_Desert");
    try
    {
        BuildGround(root.transform);
        ScatterProps(root.transform);
        BuildClouds(root.transform);
        PrefabUtility.SaveAsPrefabAsset(root, OutPath, out bool ok);
        Debug.Log(ok ? $"[DesertMap] 生成 {OutPath}" : $"[DesertMap] 保存失败 {OutPath}");
    }
    finally
    {
        Object.DestroyImmediate(root);
    }
    AssetDatabase.SaveAssets();
    AssetDatabase.Refresh();
}
// ----- 地面 -----
private static void BuildGround(Transform parent)
{
    // 实底沙地:一整块大平面(URP 沙色材质),顶面 y=0。这是真正的地面,
    // 自带 MeshCollider 供射线贴地/动物站立——不再用裂地预制体平铺(那是凹凸碎块,会露底)。
    var ground = GameObject.CreatePrimitive(PrimitiveType.Plane);
    ground.name = "Ground";
    ground.transform.SetParent(parent, false);
    ground.transform.localScale = new Vector3(GroundSize / 10f, 1f, GroundSize / 10f); // Plane 原始边长 10
    ground.GetComponent<Renderer>().sharedMaterial = GetSandMaterial();
    // 裂地只当「干裂地表」细节:压扁后稀疏铺在沙面上(含活动区),做颜色/纹理变化,不挡射击(已剥碰撞体)
    var crack = Load("Ground_Cracked");
    if (crack == null) return;
    var patches = new GameObject("CrackedPatches");
    patches.transform.SetParent(ground.transform, false);
    const int patchCount = 26;
    float half = GroundSize * 0.5f - 6f;
    for (int i = 0; i < patchCount; i++)
    {

```

```

var p = (GameObject)PrefabUtility.InstantiatePrefab(crack, patches.transform);
float s = Random.Range(3f, 6f);
p.transform.localScale = new Vector3(s, s * 0.12f, s); // 压扁成贴地的干裂斑块
p.transform.localRotation = Quaternion.Euler(0f, Random.Range(0f, 360f), 0f);
float x = Random.Range(-half, half), z = Random.Range(-half, half);
PlaceOnGround(p, new Vector3(x, 0f, z));
StripColliders(p);
}
}
/// <summary>取(没有则建)沙色 URP 材质,存为资产以便随 prefab 序列化。</summary>
private static Material GetSandMaterial()
{
    const string matDir = "Assets/Art/Map";
    const string matPath = matDir + "/DesertSand.mat";
    var existing = AssetDatabase.LoadAssetAtPath<Material>(matPath);
    if (existing != null) return existing;
    if (!Directory.Exists(matDir)) Directory.CreateDirectory(matDir);
    var shader = Shader.Find("Universal Render Pipeline/Lit");
    if (shader == null) shader = Shader.Find("Standard"); // 退路
    var mat = new Material(shader) { name = "DesertSand" };
    var sand = new Color(0.83f, 0.69f, 0.45f); // 暖沙色
    mat.SetColor("_BaseColor", sand);
    mat.SetColor("_Color", sand); // 兼容 Standard
    if (mat.HasProperty("_Smoothness")) mat.SetFloat("_Smoothness", 0.05f); // 哑光,不反光
    if (mat.HasProperty("_Glossiness")) mat.SetFloat("_Glossiness", 0.05f);
    AssetDatabase.CreateAsset(mat, matPath);
    return mat;
}
// ----- 装饰散布 -----
private static void ScatterProps(Transform parent)
{
    var props = new GameObject("Props");
    props.transform.SetParent(parent, false);
    float outer = GroundSize * 0.5f - 6f;
    foreach (var deco in Scatter)
    {
        var src = Load(deco.Name);
        if (src == null) continue;
        for (int i = 0; i < deco.Count; i++)
        {
            var go = (GameObject)PrefabUtility.InstantiatePrefab(src, props.transform);
            go.transform.localScale = Vector3.one * Random.Range(deco.MinScale, deco.MaxScale);
            go.transform.localRotation = Quaternion.Euler(0f, Random.Range(0f, 360f), 0f);
            PlaceOnGround(go, RingPos(PlayRadius, outer));
            StripColliders(go);
        }
    }
}
private static void BuildClouds(Transform parent)
{

```



```

var sky = new GameObject("Clouds");
sky.transform.SetParent(parent, false);
foreach (var deco in Clouds)
{
    var src = Load(deco.Name);
    if (src == null) continue;
    for (int i = 0; i < deco.Count; i++)
    {
        var go = (GameObject)PrefabUtility.InstantiatePrefab(src, sky.transform);
        go.transform.localScale = Vector3.one * Random.Range(deco.MinScale, deco.MaxScale);
        go.transform.localRotation = Quaternion.Euler(0f, Random.Range(0f, 360f), 0f);
        float r = Random.Range(0f, GroundSize * 0.45f);
        float a = Random.Range(0f, Mathf.PI * 2f);
        go.transform.position = new Vector3(Mathf.Cos(a) * r, Random.Range(45f, 70f), Mathf.Sin(a) * r)
;

        StripColliders(go);
    }
}
}
// ----- 工具 -----
/// <summary>把物体放到 (x,0,z),并按其包围盒底部贴到地面 y=0。</summary>
private static void PlaceOnGround(GameObject go, Vector3 xz)
{
    go.transform.position = new Vector3(xz.x, 0f, xz.z);
    var b = WorldBounds(go);
    go.transform.position += new Vector3(0f, -b.min.y, 0f);
}
/// <summary>中心活动区(inner)外、地图边(outer)内的环形随机点(只取 x/z)。</summary>
private static Vector3 RingPos(float inner, float outer)
{
    float a = Random.Range(0f, Mathf.PI * 2f);
    float r = Mathf.Sqrt(Random.Range(inner * inner, outer * outer)); // 面积均匀
    return new Vector3(Mathf.Cos(a) * r, 0f, Mathf.Sin(a) * r);
}
private static Bounds WorldBounds(GameObject go)
{
    {
        var rends = go.GetComponentInChildren<Renderer>(true);
        if (rends.Length == 0) return new Bounds(go.transform.position, Vector3.zero);
        var b = rends[0].bounds;
        for (int i = 1; i < rends.Length; i++) b.Encapsulate(rends[i].bounds);
        return b;
    }
}
private static void StripColliders(GameObject go)
{
    {
        foreach (var c in go.GetComponentInChildren<Collider>(true))
            Object.DestroyImmediate(c, true);
    }
}
private static GameObject Load(string name)
{
    var path = $"{NaturePack}/{name}.prefab";

```

```

var go = AssetDatabase.LoadAssetAtPath<GameObject>(path);
if (go == null) Debug.LogWarning($"[DesertMap] 找不到 Nature 预制体: {path}");
return go;
}
}
#endif
using System.Diagnostics;
using System.IO;
using UnityEditor;
using UnityEngine;
using Debug = UnityEngine.Debug;
/// 菜单: Tools/Network/Generate Proto
/// 调 client/Proto/gen_net_proto.bat 把 Proto/Net/*.proto 编译到 Assets/Scripts/GamePlay/Network/Messages/,
/// 完成后 AssetDatabase.Refresh() 让 Unity 重新导入生成的 .cs。Windows Editor 限定。
public static class NetProtoGenerator
{
    private const string BatRelativePath = "Proto/gen_net_proto.bat";
    private const string MenuPath = "Tools/Network/Generate Proto";
    [MenuItem(MenuPath, false, 100)]
    public static void Generate()
    {
        if (Application.platform != RuntimePlatform.WindowsEditor)
        {
            Debug.LogError("[NetProto] 仅 Windows Editor 支持 (bat 脚本)");
            return;
        }
        var projectRoot = Directory.GetParent(Application.dataPath).FullName;
        var batPath = Path.Combine(projectRoot, BatRelativePath);
        if (!File.Exists(batPath))
        {
            Debug.LogError($"[NetProto] 找不到生成脚本: {batPath}");
            return;
        }
        // UseShellExecute=false 不能直接启动 .bat, 必须经 cmd.exe。
        // /c 后整个命令用一对外层引号包住, 内部 bat 路径再用一对引号, 避免空格路径被截断。
        var psi = new ProcessStartInfo
        {
            FileName = "cmd.exe",
            Arguments = $"/c \"{batPath}\"",
            WorkingDirectory = Path.GetDirectoryName(batPath),
            UseShellExecute = false,
            RedirectStandardOutput = true,
            RedirectStandardError = true,
            CreateNoWindow = true,
        };
        Debug.Log($"[NetProto] 运行 {batPath}");
        string stdout, stderr;
        int exit;
        try
        {

```

```

        using var proc = Process.Start(psi);
        if (proc == null)
        {
            Debug.LogError("[NetProto] Process.Start 返回 null");
            return;
        }
        stdout = proc.StandardOutput.ReadToEnd();
        stderr = proc.StandardError.ReadToEnd();
        proc.WaitForExit();
        exit = proc.ExitCode;
    }
    catch (System.Exception e)
    {
        Debug.LogError($"[NetProto] 启动失败: {e}");
        return;
    }
    if (!string.IsNullOrEmpty(stdout)) Debug.Log("[NetProto] stdout:\n" + stdout.TrimEnd());
    if (!string.IsNullOrEmpty(stderr)) Debug.LogWarning("[NetProto] stderr:\n" + stderr.TrimEnd());
    if (exit != 0)
    {
        Debug.LogError($"[NetProto] 生成失败, exit={exit}");
        return;
    }
    AssetDatabase.Refresh();
    Debug.Log("[NetProto] 完成, AssetDatabase 已刷新");
}
}

#if UNITY_EDITOR
using System.IO;
using TMPro;
using UnityEditor;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// 一键生成网格背包 UI 预制体到 Resources/UI/Bag 下,供 UIMgr/ResMgr 按路径加载。
/// 程序化构建:Unity 自动解析脚本 GUID / TMP 字体,比手写 .prefab YAML 可靠;脚本字段引用在此接好。
///
/// 菜单:Tools/Bag/Build Bag UI Prefabs
/// 产物:
/// - BagItem.prefab    物品控件(BagItemWidget)
/// - BagPanel.prefab   纯背包面板(BagPanel:单网格)
/// - InteractPrompt.prefab 世界交互提示
///
/// 共享 UI(右键菜单/拆分弹窗/tooltip)挂在 <see cref="GridHostPanelBase"/> 字段上。
/// </summary>
public static class BagPrefabBuilder
{
    private const string Dir = "Assets/Resources/UI/Bag";
    private const string ItemPrefabPath = Dir + "/BagItem.prefab";
    private const string BagPanelPath = Dir + "/BagPanel.prefab";

```

```

private const string PromptPath = Dir + "/InteractPrompt.prefab";
private const string FontPath = "Assets/Art/Fonts/SIMHEI SDF.asset";
private const float Cell = 80f;
private const float Gap = 4f;
private static TMP_FontAsset _font;
[MenuItem("Tools/Bag/Build Bag UI Prefabs")]
public static void BuildAll()
{
    if (!Directory.Exists(Dir)) Directory.CreateDirectory(Dir);
    _font = AssetDatabase.LoadAssetAtPath<TMP_FontAsset>(FontPath);
    if (_font == null) Debug.LogWarning($"[BagPrefabBuilder] 未找到字体 {FontPath},回退默认字体。");
    BuildItemPrefab();
    AssetDatabase.SaveAssets();
    AssetDatabase.Refresh();
    var itemPrefab = AssetDatabase.LoadAssetAtPath<GameObject>(ItemPrefabPath);
    BuildBagPanel(itemPrefab);
    BuildPromptPrefab();
    AssetDatabase.SaveAssets();
    AssetDatabase.Refresh();
    Debug.Log($"[BagPrefabBuilder] Done. Prefabs at {Dir}");
}
// ===== 物品控件 =====
private static void BuildItemPrefab()
{
    var root = NewUI("BagItem", out var rootRt);
    rootRt.sizeDelta = new Vector2(Cell, Cell);
    root.AddComponent<BagItemWidget>();
    PrefabUtility.SaveAsPrefabAsset(root, ItemPrefabPath);
    Object.DestroyImmediate(root);
    Debug.Log($"[BagPrefabBuilder] Built {ItemPrefabPath}");
}
// ===== 纯背包面板 =====
private static void BuildBagPanel(GameObject itemPrefab)
{
    // 背包网格 10×8 = 800×640px。窗口 920×900,网格区(去边距)820×700 容纳之,不与标题/按钮重叠。
    var root = MakeWindowRoot("BagPanel", new Vector2(920, 900), out var window);
    Title(window, "背包 (左键使用 / 右键菜单 / 拖拽移动)");
    var capGo = NewUI("CapacityText", out var capRt, window.transform);
    capRt.anchorMin = new Vector2(1, 1); capRt.anchorMax = new Vector2(1, 1); capRt.pivot = new Vector2(1,
1);
    capRt.anchoredPosition = new Vector2(-110, -22); capRt.sizeDelta = new Vector2(200, 36);
    var capText = NewText(capGo, "0/80", 24, TextAlignmentOptions.Right);
    var closeBtn = BuildButton("CloseBtn", "X", window.transform,
        new Vector2(1, 1), new Vector2(-36, -36), new Vector2(56, 56), new Color(0.5f, 0.2f, 0.2f, 1f));
    var sortBtn = BuildButton("SortBtn", "整理", window.transform,
        new Vector2(0.5f, 0), new Vector2(0, 42), new Vector2(180, 60), new Color(0.2f, 0.4f, 0.55f, 1f));
    // 单网格区:上留 90(标题) 下留 110(整理按钮),BagGridView 居中其内
    var gridArea = NewUI("GridArea", out var gaRt, window.transform);
    gaRt.anchorMin = new Vector2(0, 0); gaRt.anchorMax = new Vector2(1, 1);
    gaRt.offsetMin = new Vector2(50, 110); gaRt.offsetMax = new Vector2(-50, -90);

```

```

var bagGrid = NewUI("BagGrid", out _, gridArea.transform).AddComponent<BagGridView>();
Stretch((RectTransform)bagGrid.transform, 0);
var panel = root.AddComponent<BagPanel>();
var cg = root.GetComponent<CanvasGroup>();
panel.canvasGroup = cg;
panel.uiType = UIEnum.BagPanel;
panel.bagGrid = bagGrid;
panel.sortBtn = sortBtn;
panel.closeBtn = closeBtn;
panel.capacityText = capText;
ApplyHostCommon(panel, itemPrefab, root.transform);
PrefabUtility.SaveAsPrefabAsset(root, BagPanelPath);
Object.DestroyImmediate(root);
Debug.Log($"[BagPrefabBuilder] Built {BagPanelPath}");
}
// ===== 交互提示 =====
private static void BuildPromptPrefab()
{
    var root = NewUI("InteractPrompt", out var rootRt);
    rootRt.anchorMin = new Vector2(0.5f, 0); rootRt.anchorMax = new Vector2(0.5f, 0); rootRt.pivot = new Vector2(0.5f, 0);
    rootRt.anchoredPosition = new Vector2(0, 180); rootRt.sizeDelta = new Vector2(360, 56);
    root.AddComponent<Image>().color = new Color(0f, 0f, 0f, 0.6f);
    // 文本作为子物体(一个 GameObject 只能有一个 Graphic,不能 Image+TMP 同挂)
    var labelGo = NewUI("Label", out var labelRt, root.transform);
    Stretch(labelRt, 0);
    NewText(labelGo, "按 F 打开宝箱", 26, TextAlignmentOptions.Center);
    PrefabUtility.SaveAsPrefabAsset(root, PromptPath);
    Object.DestroyImmediate(root);
    Debug.Log($"[BagPrefabBuilder] Built {PromptPath}");
}
// ===== 共享 UI(挂到 GridHostPanelBase)=====
/// <summary>设置 host 基类公共字段 + 构建右键菜单/拆分弹窗/tooltip。</summary>
private static void ApplyHostCommon(GridHostPanelBase panel, GameObject itemPrefab, Transform root)
{
    panel.itemWidgetPrefab = itemPrefab;
    panel.cellSize = Cell;
    panel.cellGap = Gap;
    panel.uiFont = _font;
    BuildContextMenu(root, panel);
    BuildSplitDialog(root, panel);
    BuildTooltip(root, panel);
}
private static void BuildContextMenu(Transform parent, GridHostPanelBase panel)
{
    var menu = NewUI("ContextMenu", out var menuRt, parent);
    Stretch(menuRt, 0);
    var blockerImg = menu.AddComponent<Image>();
    blockerImg.color = new Color(0, 0, 0, 0.01f);
    var blockerBtn = menu.AddComponent<Button>();

```

```

blockerBtn.targetGraphic = blockerImg;
var box = NewUI("Panel", out var boxRt, menu.transform);
boxRt.pivot = new Vector2(0, 1);
boxRt.sizeDelta = new Vector2(160, 232);
box.AddComponent<Image>().color = new Color(0.16f, 0.17f, 0.2f, 0.98f);
var use = BuildStackedButton("UseBtn", "使用", box.transform, 0, new Color(0.25f, 0.4f, 0.3f, 1f));
var rotate = BuildStackedButton("RotateBtn", "旋转", box.transform, 1, new Color(0.3f, 0.4f, 0.5f, 1f))
;
var split = BuildStackedButton("SplitBtn", "拆分", box.transform, 2, new Color(0.3f, 0.35f, 0.5f, 1f));
var discard = BuildStackedButton("DiscardBtn", "丢弃", box.transform, 3, new Color(0.5f, 0.25f, 0.25f,
1f));
panel.contextMenu = menu;
panel.contextMenuPanel = boxRt;
panel.ctxUseBtn = use;
panel.ctxRotateBtn = rotate;
panel.ctxSplitBtn = split;
panel.ctxDiscardBtn = discard;
}
private static Button BuildStackedButton(string name, string label, Transform parent, int index, Color colo
r)
{
var go = NewUI(name, out var rt, parent);
rt.anchorMin = new Vector2(0, 1); rt.anchorMax = new Vector2(1, 1); rt.pivot = new Vector2(0.5f, 1);
rt.offsetMin = new Vector2(8, 0); rt.offsetMax = new Vector2(-8, 0);
rt.sizeDelta = new Vector2(rt.sizeDelta.x, 52);
rt.anchoredPosition = new Vector2(0, -8 - index * 56);
var img = go.AddComponent<Image>();
img.color = color;
var btn = go.AddComponent<Button>();
btn.targetGraphic = img;
var labelGo = NewUI("Label", out var labelRt, go.transform);
Stretch(labelRt, 0);
NewText(labelGo, label, 24, TextAlignmentOptions.Center);
return btn;
}
private static void BuildSplitDialog(Transform parent, GridHostPanelBase panel)
{
var dlg = NewUI("SplitDialog", out var dlgRt, parent);
Stretch(dlgRt, 0);
dlg.AddComponent<Image>().color = new Color(0, 0, 0, 0.5f);
var box = NewUI("Panel", out var boxRt, dlg.transform);
boxRt.anchorMin = boxRt.anchorMax = boxRt.pivot = new Vector2(0.5f, 0.5f);
boxRt.sizeDelta = new Vector2(420, 240);
box.AddComponent<Image>().color = new Color(0.16f, 0.17f, 0.2f, 0.99f);
var titleGo = NewUI("Title", out var titleRt, box.transform);
titleRt.anchorMin = new Vector2(0, 1); titleRt.anchorMax = new Vector2(1, 1); titleRt.pivot = new Vecto
r2(0.5f, 1);
titleRt.anchoredPosition = new Vector2(0, -14); titleRt.sizeDelta = new Vector2(-24, 40);
NewText(titleGo, "拆分堆叠", 26, TextAlignmentOptions.Center);
var amtGo = NewUI("Amount", out var amtRt, box.transform);

```

```

amtRt.anchorMin = new Vector2(0, 1); amtRt.anchorMax = new Vector2(1, 1); amtRt.pivot = new Vector2(0.5
f, 1);
amtRt.anchoredPosition = new Vector2(0, -64); amtRt.sizeDelta = new Vector2(-24, 36);
var amtText = NewText(amtGo, "拆出 1 / 留 1", 22, TextAlignmentOptions.Center);
var slider = BuildSlider("Slider", box.transform, new Vector2(0, 36), new Vector2(360, 30));
var confirm = BuildButton("Confirm", "确认", box.transform,
    new Vector2(0.5f, 0), new Vector2(-100, 30), new Vector2(160, 54), new Color(0.25f, 0.45f, 0.3f, 1f
));
var cancel = BuildButton("Cancel", "取消", box.transform,
    new Vector2(0.5f, 0), new Vector2(100, 30), new Vector2(160, 54), new Color(0.45f, 0.3f, 0.3f, 1f)
);
panel.splitDialog = dlg;
panel.splitSlider = slider;
panel.splitAmountText = amtText;
panel.splitConfirmBtn = confirm;
panel.splitCancelBtn = cancel;
}
private static Slider BuildSlider(string name, Transform parent, Vector2 anchoredPos, Vector2 size)
{
    var go = NewUI(name, out var rt, parent);
    rt.anchorMin = new Vector2(0.5f, 0); rt.anchorMax = new Vector2(0.5f, 0); rt.pivot = new Vector2(0.5f,
0);
    rt.anchoredPosition = anchoredPos;
    rt.sizeDelta = size;
    var slider = go.AddComponent<Slider>();
    var bg = NewUI("Background", out var bgRt, go.transform);
    bgRt.anchorMin = new Vector2(0, 0.25f); bgRt.anchorMax = new Vector2(1, 0.75f);
    bgRt.offsetMin = Vector2.zero; bgRt.offsetMax = Vector2.zero;
    bg.AddComponent<Image>().color = new Color(0.3f, 0.3f, 0.35f, 1f);
    var fillArea = NewUI("Fill Area", out var faRt, go.transform);
    faRt.anchorMin = new Vector2(0, 0.25f); faRt.anchorMax = new Vector2(1, 0.75f);
    faRt.offsetMin = new Vector2(8, 0); faRt.offsetMax = new Vector2(-8, 0);
    var fill = NewUI("Fill", out var fillRt, fillArea.transform);
    fillRt.anchorMin = new Vector2(0, 0); fillRt.anchorMax = new Vector2(0, 1);
    fillRt.sizeDelta = new Vector2(10, 0);
    fill.AddComponent<Image>().color = new Color(0.4f, 0.6f, 0.9f, 1f);
    var handleArea = NewUI("Handle Slide Area", out var haRt, go.transform);
    haRt.anchorMin = Vector2.zero; haRt.anchorMax = Vector2.one;
    haRt.offsetMin = new Vector2(8, 0); haRt.offsetMax = new Vector2(-8, 0);
    var handle = NewUI("Handle", out var handleRt, handleArea.transform);
    handleRt.sizeDelta = new Vector2(22, 0);
    var handleImg = handle.AddComponent<Image>();
    handleImg.color = Color.white;
    slider.fillRect = fillRt;
    slider.handleRect = handleRt;
    slider.targetGraphic = handleImg;
    slider.direction = Slider.Direction.LeftToRight;
    slider.wholeNumbers = true;
    slider.minValue = 1;
    slider.maxValue = 2;

```

```

    slider.value = 1;
    return slider;
}

private static void BuildTooltip(Transform parent, GridHostPanelBase panel)
{
    var tip = NewUI("Tooltip", out var tipRt, parent);
    Stretch(tipRt, 0);
    var box = NewUI("Panel", out var boxRt, tip.transform);
    boxRt.pivot = new Vector2(0, 1);
    boxRt.sizeDelta = new Vector2(320, 168);
    var boxImg = box.AddComponent<Image>();
    boxImg.color = new Color(0.08f, 0.09f, 0.11f, 0.96f);
    boxImg.raycastTarget = false;
    var nameGo = NewUI("Name", out var nameRt, box.transform);
    nameRt.anchorMin = new Vector2(0, 1); nameRt.anchorMax = new Vector2(1, 1); nameRt.pivot = new Vector2(
0.5f, 1);
    nameRt.anchoredPosition = new Vector2(0, -10); nameRt.sizeDelta = new Vector2(-24, 34);
    var nameText = NewText(nameGo, "", 24, TextAlignmentOptions.TopLeft);
    var descGo = NewUI("Desc", out var descRt, box.transform);
    descRt.anchorMin = new Vector2(0, 1); descRt.anchorMax = new Vector2(1, 1); descRt.pivot = new Vector2(
0.5f, 1);
    descRt.anchoredPosition = new Vector2(0, -50); descRt.sizeDelta = new Vector2(-24, 78);
    var descText = NewText(descGo, "", 18, TextAlignmentOptions.TopLeft);
    descText.color = new Color(0.8f, 0.8f, 0.82f, 1f);
    descText.enableWordWrapping = true;
    descText.overflowMode = TextOverflowModes.Truncate;
    var valGo = NewUI("Value", out var valRt, box.transform);
    valRt.anchorMin = new Vector2(0, 0); valRt.anchorMax = new Vector2(1, 0); valRt.pivot = new Vector2(0.5
f, 0);
    valRt.anchoredPosition = new Vector2(0, 10); valRt.sizeDelta = new Vector2(-24, 28);
    var valText = NewText(valGo, "", 18, TextAlignmentOptions.BottomLeft);
    valText.color = new Color(0.95f, 0.85f, 0.4f, 1f);
    panel.tooltip = tip;
    panel.tooltipPanel = boxRt;
    panel.tipNameText = nameText;
    panel.tipDescText = descText;
    panel.tipValueText = valText;
}

// ===== 工具 =====
/// <summary>建一个 UIPageBase 标准根(全屏遮罩 + CanvasGroup + YOTOUIShow)+ 居中窗口,返回 root,out window。
</summary>
private static GameObject MakeWindowRoot(string name, Vector2 windowSize, out GameObject window)
{
    var root = NewUI(name, out var rootRt);
    Stretch(rootRt, 0);
    root.AddComponent<CanvasGroup>();
    root.AddComponent<YOTOUIShow>();
    var dim = root.AddComponent<Image>();
    dim.color = new Color(0f, 0f, 0f, 0.6f);
    window = NewUI("Window", out var winRt, root.transform);

```



```

winRt.anchorMin = winRt.anchorMax = winRt.pivot = new Vector2(0.5f, 0.5f);
winRt.sizeDelta = windowSize;
winRt.anchoredPosition = Vector2.zero;
window.AddComponent<Image>().color = new Color(0.12f, 0.13f, 0.16f, 0.96f);
return root;
}

private static void Title(GameObject window, string text)
{
    var titleGo = NewUI("Title", out var titleRt, window.transform);
    titleRt.anchorMin = new Vector2(0, 1); titleRt.anchorMax = new Vector2(0.5f, 1); titleRt.pivot = new Vector2(0, 1);
    titleRt.anchoredPosition = new Vector2(20, -16); titleRt.sizeDelta = new Vector2(-40, 44);
    NewText(titleGo, text, 26, TextAlignmentOptions.Left);
}

private static GameObject NewUI(string name, out RectTransform rt, Transform parent = null)
{
    var go = new GameObject(name, typeof(RectTransform));
    rt = go.GetComponent<RectTransform>();
    if (parent != null) rt.SetParent(parent, false);
    return go;
}

private static void Stretch(RectTransform rt, float inset)
{
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = new Vector2(inset, inset);
    rt.offsetMax = new Vector2(-inset, -inset);
}

private static TextMeshProUGUI NewText(GameObject go, string text, float size, TextAlignmentOptions align)
{
    var tmp = go.AddComponent<TextMeshProUGUI>();
    tmp.text = text;
    tmp.fontSize = UITheme.Font(size);
    tmp.alignment = align;
    tmp.color = Color.white;
    tmp.raycastTarget = false;
    var font = _font != null ? _font : TMP_Settings.defaultFontAsset;
    if (font != null) tmp.font = font;
    return tmp;
}

private static Button BuildButton(string name, string label, Transform parent,
    Vector2 anchor, Vector2 anchoredPos, Vector2 size, Color color)
{
    var go = NewUI(name, out var rt, parent);
    rt.anchorMin = rt.anchorMax = rt.pivot = anchor;
    rt.anchoredPosition = anchoredPos;
    rt.sizeDelta = size;
    var img = go.AddComponent<Image>();
    img.color = color;
    var btn = go.AddComponent<Button>();

```

```

        btn.targetGraphic = img;
        var labelGo = NewUI("Label", out var labelRt, go.transform);
        Stretch(labelRt, 0);
        NewText(labelGo, label, 26, TextAlignmentOptions.Center);
        return btn;
    }
}
#endif
#if UNITY_EDITOR
using System;
using System.Collections.Generic;
using UnityEditor;
using UnityEngine;
/// <summary>
/// 一键重建所有 UI 预制体。各面板预制体里的字号由各 *Builder 烘焙,改了 <see cref="UITheme.FontScale"/>(或任何
Builder 代码)后,
/// 已生成的 .prefab 不会自动更新,需重跑对应 <c>Tools/UI/Build *** Prefab</c>。本菜单把它们串起来一次跑完,省去
逐个点。
/// 单个 Builder 抛异常不会中断整批:记录失败项,最后汇总并统一刷新资源库。
/// </summary>
public static class BuildAllUIPrefabs
{
    /// <summary>(名字, 构建委托) 清单——新增面板时在这里补一行即可纳入一键重建。</summary>
    private static readonly (string name, Action build)[] Builders =
    {
        ("CodexPanel",    CodexPanelBuilder.Build),
        ("ConfirmPanel",  ConfirmPanelBuilder.Build),
        ("EquipPanel",    EquipPanelBuilder.Build),
        ("FinishPanel",   FinishPanelBuilder.Build),
        ("LeaderboardPanel", LeaderboardPanelBuilder.Build),
        ("GameMainPanel", GameMainPanelBuilder.Build),
        ("LoadingPanel",  LoadingPanelBuilder.Build),
        ("LoginPanel",    LoginPanelBuilder.Build),
        ("MapSelectPanel", MapSelectPanelBuilder.Build),
        ("RewardClaimPanel", RewardClaimPanelBuilder.Build),
        ("SaveSlotPanel", SaveSlotPanelBuilder.Build),
        ("SettingPanel",  SettingPanelBuilder.Build),
        ("ShopPanel",     ShopPanelBuilder.Build),
        ("StartPanel",    StartPanelBuilder.Build),
        ("TaskPanel",     TaskPanelBuilder.Build),
        ("Bag UI",        BagPrefabBuilder.BuildAll),
    };
    [MenuItem("Tools/UI/Build ALL UI Prefabs", priority = 0)]
    public static void BuildAll()
    {
        var failed = new List<string>();
        int total = Builders.Length;
        try
        {
            for (int i = 0; i < total; i++)

```

```

{
    var (name, build) = Builders[i];
    EditorUtility.DisplayProgressBar("Build ALL UI Prefabs",
        $"({i + 1}/{total}) {name}", (float)i / total);
    try
    {
        build();
        Debug.Log($"[BuildAllUIPrefabs] [OK] {name}");
    }
    catch (Exception e)
    {
        failed.Add(name);
        Debug.LogError($"[BuildAllUIPrefabs] [FAIL] {name} 构建失败:{e}");
    }
}
}
finally
{
    EditorUtility.ClearProgressBar();
}
AssetDatabase.SaveAssets();
AssetDatabase.Refresh();
if (failed.Count == 0)
    Debug.Log($"[BuildAllUIPrefabs] 全部完成,共 {total} 个 UI 预制体已重建。");
else
    Debug.LogError($"[BuildAllUIPrefabs] 完成 {total - failed.Count}/{total},失败:{string.Join(", ", failed)}");
}
}
#endif
#if UNITY_EDITOR
using System.IO;
using TMPro;
using UnityEditor;
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// 一键生成图鉴界面预制体 CodexPanel.prefab 到 Resources/UI/Codex 下,供 UIMgr/ResMgr 按路径加载。
/// 只搭外壳:浅紫背景 + 返回 / 资源金币(绿色胶囊) + 装饰公仔/荣誉卡片 两页签 + 居中内容面板(2 列网格容器) + 底部分页条。
/// 卡片(图片 + 解锁徽标)由 <see cref="CodexPanel"/> 运行时按配表 <c>codexConfig</c> 分页构建。
///
/// 尺寸按"画布宽恒为 1920 单位"(CanvasScaler match=width)给,竖屏四边留边自适应。
/// 菜单:Tools/UI/Build CodexPanel Prefab
/// </summary>
public static class CodexPanelBuilder
{
    private const string Dir = "Assets/Resources/UI/Codex";
    private const string PrefabPath = Dir + "/CodexPanel.prefab";
    private const string ButtonPrefabPath = "Assets/Resources/UI/Common/CommonButton.prefab";

```

```

private const string FontPath = "Assets/Art/Fonts/SIMHEI SDF.asset";
private static TMP_FontAsset _font;
private static GameObject _btnPrefab;
[MenuItem("Tools/UI/Build CodexPanel Prefab")]
public static void Build()
{
    if (!Directory.Exists(Dir)) Directory.CreateDirectory(Dir);
    _font = AssetDatabase.LoadAssetAtPath<TMP_FontAsset>(FontPath);
    _btnPrefab = AssetDatabase.LoadAssetAtPath<GameObject>(ButtonPrefabPath);
    if (_btnPrefab == null)
    {
        Debug.LogError($"[CodexPanelBuilder] 未找到通用按钮 {ButtonPrefabPath},无法生成。");
        return;
    }
    // ----- 根(浅紫背景 + CanvasGroup + YOTOUIShow + CodexPanel)-----
    var root = NewUI("CodexPanel", out var rootRt);
    Stretch(rootRt);
    root.AddComponent<Image>().color = new Color(0.91f, 0.90f, 0.95f, 1f); // 浅紫底
    var cg = root.AddComponent<CanvasGroup>();
    root.AddComponent<YOTOUIShow>();
    // ----- 顶部:返回(左上)-----
    var backBtn = BuildButton("Btn_Back", "返回", root.transform, 44);
    var backRt = (RectTransform)backBtn.transform;
    backRt.anchorMin = backRt.anchorMax = new Vector2(0, 1); backRt.pivot = new Vector2(0, 1);
    backRt.anchoredPosition = new Vector2(40, -40); backRt.sizeDelta = new Vector2(200, 90);
    // ----- 顶部:资源金币(绿色胶囊,返回按钮右侧)-----
    var coinGo = NewUI("Coin", out var coinRt, root.transform);
    coinRt.anchorMin = coinRt.anchorMax = new Vector2(0, 1); coinRt.pivot = new Vector2(0, 1);
    coinRt.anchoredPosition = new Vector2(280, -48); coinRt.sizeDelta = new Vector2(820, 96); // 加宽:要容
    下「金币 + 图鉴 x/y」两段,原 340 文字会溢出盖住返回键
    var coinBg = coinGo.AddComponent<Image>();
    coinBg.sprite = BuiltinSprite("UI/Skin/UISprite.psd");
    coinBg.type = Image.Type.Sliced;
    coinBg.color = new Color(0.56f, 0.78f, 0.30f, 1f); // 绿色胶囊
    var coinText = NewChildText(coinGo, "Value", "0", 40, TextAlignmentOptions.Right);
    ((RectTransform)coinText.transform).offsetMax = new Vector2(-28, 0); // 右侧留边距
    UICurrencyPill.AddIconLeft(coinGo, (RectTransform)coinText.transform, UICurrencyPill.IconGold); // 金币
    用图标,不写文字
    // 动物图鉴是单分类,「装饰公仔 / 荣誉卡片」两个页签已停用——不再生成,空出的顶部空间让内容面板上移。
    // (CodexPanel 的 tabDoll/tabCard 字段保持为空,OnLoad 里对空引用已做保护。)
    // ----- 中部:内容面板(2 列网格容器)-----
    var panelGo = NewUI("Panel", out var panelRt, root.transform);
    panelRt.anchorMin = Vector2.zero; panelRt.anchorMax = Vector2.one;
    // 顶部上移到顶栏正下方(原 -360 是给页签留的位,页签已移除,否则顶部空一大块、网格整体偏低)
    panelRt.offsetMin = new Vector2(50, 360); panelRt.offsetMax = new Vector2(-50, -170);
    var panelBg = panelGo.AddComponent<Image>();
    panelBg.sprite = BuiltinSprite("UI/Skin/UISprite.psd");
    panelBg.type = Image.Type.Sliced;
    panelBg.color = new Color(0.86f, 0.84f, 0.93f, 1f); // 内容面板浅紫
    var gridGo = NewUI("Grid", out var gridRt, panelGo.transform);

```

```

gridRt.anchorMin = Vector2.zero; gridRt.anchorMax = Vector2.one;
gridRt.offsetMin = new Vector2(30, 30); gridRt.offsetMax = new Vector2(-30, -30);
var glg = gridGo.AddComponent<GridLayoutGroup>();
glg.constraint = GridLayoutGroup.Constraint.FixedColumnCount;
glg.constraintCount = 2;
glg.cellSize = new Vector2(820, 760);
glg.spacing = new Vector2(40, 40);
glg.padding = new RectOffset(30, 30, 30, 30);
glg.childAlignment = TextAnchor.UpperCenter;
// ----- 底部:分页条(首页 / 上一页 / 页码 / 下一页 / 末页)-----
var pager = NewUI("Pager", out var pagerRt, root.transform);
pagerRt.anchorMin = new Vector2(0, 0); pagerRt.anchorMax = new Vector2(1, 0); pagerRt.pivot = new Vector2(0.5f, 0);
pagerRt.offsetMin = new Vector2(50, 140); pagerRt.offsetMax = new Vector2(-50, 300);
var pagerHlg = pager.AddComponent<HorizontalLayoutGroup>();
pagerHlg.spacing = 24;
pagerHlg.childAlignment = TextAnchor.MiddleCenter;
pagerHlg.childControlWidth = true; pagerHlg.childControlHeight = true;
pagerHlg.childForceExpandWidth = false; pagerHlg.childForceExpandHeight = false;
var btnFirst = BuildNavButton("Btn_First", "<", pager.transform);
var btnPrev = BuildNavButton("Btn_Prev", "<", pager.transform);
var pageText = BuildPageText(pager.transform);
var btnNext = BuildNavButton("Btn_Next", ">", pager.transform);
var btnLast = BuildNavButton("Btn_Last", ">", pager.transform);
// ----- 接脚本字段 -----
var panel = root.AddComponent<CodexPanel>();
panel.canvasGroup = cg;
panel.uiType = UIEnum.CodexPanel;
panel.backBtn = backBtn;
panel.coinText = coinText;
// tabDoll / tabCard 不再生成,留空(动物图鉴单分类)
panel.grid = gridRt;
panel.pageText = pageText;
panel.btnFirst = btnFirst;
panel.btnPrev = btnPrev;
panel.btnNext = btnNext;
panel.btnLast = btnLast;
PrefabUtility.SaveAsPrefabAsset(root, PrefabPath);
Object.DestroyImmediate(root);
AssetDatabase.SaveAssets();
AssetDatabase.Refresh();
Debug.Log($"[CodexPanelBuilder] Built {PrefabPath}");
}
// ===== 构件 =====
/// <summary>分页方向按钮:固定首选尺寸的小方按钮。</summary>
private static Button BuildNavButton(string name, string label, Transform parent)
{
    var btn = BuildButton(name, label, parent, 48);
    var le = btn.gameObject.AddComponent<LayoutElement>();
    le.preferredWidth = 150; le.preferredHeight = 110;

```

```

    return btn;
}
/// <summary>页码文本:夹在方向按钮中间,深色显示在浅底上。</summary>
private static TextMeshProUGUI BuildPageText(Transform parent)
{
    var go = NewUI("PageText", out var rt, parent);
    var le = go.AddComponent<LayoutElement>();
    le.preferredWidth = 260; le.preferredHeight = 110;
    return NewText(go, "1 / 1", 52, TextAlignmentOptions.Center, new Color(0.25f, 0.24f, 0.30f, 1f));
}
private static Button BuildButton(string name, string label, Transform parent, float fontSize)
{
    var go = (GameObject)PrefabUtility.InstantiatePrefab(_btnPrefab, parent);
    go.name = name;
    go.SetActive(true);
    var text = go.GetComponentInChildren<TextMeshProUGUI>(true);
    if (text != null) { text.text = label; text.fontSize = UITheme.Font(fontSize); }
    return go.GetComponent<Button>();
}
// ===== 工具 =====
private static GameObject NewUI(string name, out RectTransform rt, Transform parent = null)
{
    var go = new GameObject(name, typeof(RectTransform));
    rt = go.GetComponent<RectTransform>();
    if (parent != null) rt.SetParent(parent, false);
    return go;
}
private static void Stretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero; rt.anchorMax = Vector2.one;
    rt.offsetMin = Vector2.zero; rt.offsetMax = Vector2.zero;
}
private static TextMeshProUGUI NewChildText(GameObject parent, string name, string text, float size, TextAl
ignmentOptions align)
{
    var go = NewUI(name, out var rt, parent.transform);
    Stretch(rt);
    return NewText(go, text, size, align, Color.white);
}
private static TextMeshProUGUI NewText(GameObject go, string text, float size, TextAlignmentOptions align,
Color color)
{
    var tmp = go.AddComponent<TextMeshProUGUI>();
    tmp.text = text; tmp.fontSize = UITheme.Font(size); tmp.alignment = align; tmp.color = color; tmp.raycastTarget = false;
    var font = _font != null ? _font : TMP_Settings.defaultFontAsset;
    if (font != null) tmp.font = font;
    return tmp;
}
private static Sprite BuiltinSprite(string path)

```

```

{
    return AssetDatabase.GetBuiltinExtraResource<Sprite>(path);
}
}
#endif
#if UNITY_EDITOR
using System.IO;
using TMPro;
using UnityEditor;
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// 一键生成通用确认弹窗预制体 ConfirmPanel.prefab 到 Resources/UI/Common 下。
/// 结构:全屏遮罩 + 居中窗口 + 标题 + 正文 + 取消/确认两个按钮。脚本字段在此接好。
///
/// 菜单:Tools/UI/Build ConfirmPanel Prefab
/// </summary>
public static class ConfirmPanelBuilder
{
    private const string Dir = "Assets/Resources/UI/Common";
    private const string PrefabPath = Dir + "/ConfirmPanel.prefab";
    private const string FontPath = "Assets/Art/Fonts/SIMHEI SDF.asset";
    private static TMP_FontAsset _font;
    [MenuItem("Tools/UI/Build ConfirmPanel Prefab")]
    public static void Build()
    {
        if (!Directory.Exists(Dir)) Directory.CreateDirectory(Dir);
        _font = AssetDatabase.LoadAssetAtPath<TMP_FontAsset>(FontPath);
        // 根:全屏遮罩(挡住下层点击)
        var root = NewUI("ConfirmPanel", out var rootRt);
        Stretch(rootRt);
        root.AddComponent<Image>().color = new Color(0f, 0f, 0f, 0.6f);
        var cg = root.AddComponent<CanvasGroup>();
        root.AddComponent<YOTOUIShow>();
        // 窗口(竖屏适配:画布宽恒 1920 单位,窗口取较宽较高的尺寸,在手机竖屏上才不至于又小又扁)
        var window = NewUI("Window", out var winRt, root.transform);
        winRt.anchorMin = winRt.anchorMax = winRt.pivot = new Vector2(0.5f, 0.5f);
        winRt.sizeDelta = new Vector2(1120, 760);
        window.AddComponent<Image>().color = new Color(0.13f, 0.14f, 0.17f, 0.99f);
        // 标题(顶部,带高 120 容下 40pt×2)
        var titleGo = NewUI("Title", out var titleRt, window.transform);
        titleRt.anchorMin = new Vector2(0, 1); titleRt.anchorMax = new Vector2(1, 1); titleRt.pivot = new Vector2(0.5f, 1);
        titleRt.anchoredPosition = new Vector2(0, -30); titleRt.sizeDelta = new Vector2(-60, 120);
        var titleText = NewText(titleGo, "提示", 40, TextAlignmentOptions.Center);
        // 正文(居中,自动换行;上让出标题、下让出按钮)
        var msgGo = NewUI("Message", out var msgRt, window.transform);
        msgRt.anchorMin = new Vector2(0, 0); msgRt.anchorMax = new Vector2(1, 1);
        msgRt.offsetMin = new Vector2(50, 230); msgRt.offsetMax = new Vector2(-50, -160);
        var msgText = NewText(msgGo, "", 32, TextAlignmentOptions.Center);
    }
}

```

```

msgText.enableWordWrapping = true;
// 取消 / 确认(底部并排,加大触控面积)
var cancelBtn = BuildButton("CancelBtn", "取消", window.transform,
    new Vector2(0.5f, 0), new Vector2(-290, 50), new Vector2(480, 150), new Color(0.3f, 0.32f, 0.4f, 1f
), out var cancelLabel);
var confirmBtn = BuildButton("ConfirmBtn", "确认", window.transform,
    new Vector2(0.5f, 0), new Vector2(290, 50), new Vector2(480, 150), new Color(0.55f, 0.28f, 0.28f, 1
f), out var confirmLabel);
// 接脚本字段
var panel = root.AddComponent<ConfirmPanel>();
panel.canvasGroup = cg;
panel.uiType = UIEnum.ConfirmPanel;
panel.titleText = titleText;
panel.messageText = msgText;
panel.confirmBtn = confirmBtn;
panel.cancelBtn = cancelBtn;
panel.confirmLabel = confirmLabel;
panel.cancelLabel = cancelLabel;
PrefabUtility.SaveAsPrefabAsset(root, PrefabPath);
Object.DestroyImmediate(root);
AssetDatabase.SaveAssets();
AssetDatabase.Refresh();
Debug.Log($"[ConfirmPanelBuilder] Built {PrefabPath}");
}
// ===== 工具 =====
private static GameObject NewUI(string name, out RectTransform rt, Transform parent = null)
{
    var go = new GameObject(name, typeof(RectTransform));
    rt = go.GetComponent<RectTransform>();
    if (parent != null) rt.SetParent(parent, false);
    return go;
}
private static void Stretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero; rt.anchorMax = Vector2.one;
    rt.offsetMin = Vector2.zero; rt.offsetMax = Vector2.zero;
}
private static TextMeshProUGUI NewText(GameObject go, string text, float size, TextAlignmentOptions align)
{
    var tmp = go.AddComponent<TextMeshProUGUI>();
    tmp.text = text;
    tmp.fontSize = UITheme.Font(size);
    tmp.alignment = align;
    tmp.color = Color.white;
    tmp.raycastTarget = false;
    var font = _font != null ? _font : TMP_Settings.defaultFontAsset;
    if (font != null) tmp.font = font;
    return tmp;
}
private static Button BuildButton(string name, string label, Transform parent,

```



```

Vector2 anchor, Vector2 anchoredPos, Vector2 size, Color color, out TextMeshProUGUI labelText)
{
    var go = NewUI(name, out var rt, parent);
    rt.anchorMin = rt.anchorMax = rt.pivot = anchor;
    rt.anchoredPosition = anchoredPos;
    rt.sizeDelta = size;
    var img = go.AddComponent<Image>();
    img.color = color;
    var btn = go.AddComponent<Button>();
    btn.targetGraphic = img;
    var lblGo = NewUI("Label", out var lblRt, go.transform);
    Stretch(lblRt);
    labelText = NewText(lblGo, label, 34, TextAlignmentOptions.Center);
    return btn;
}
}
#endif
#if UNITY_EDITOR
using System.IO;
using TMPPro;
using UnityEditor;
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// 一键生成装备界面预制体 EquipPanel.prefab 到 Resources/UI/Equip 下。
/// 外壳:全屏遮罩 + 居中窗口(返回 / 资源金币 / 标题「选择你的装备」 + 枪械/瞄准镜/子弹 三个横向滚动行 + 出发)。
/// 每行的装备卡由 <see cref="EquipPanel"/> 运行时按 <see cref="LoadoutSystem.CategoryItems"/> 构建。
///
/// 尺寸按"画布宽恒为 1920 单位"(CanvasScaler match=width)给,竖屏四边留边自适应。
/// 菜单:Tools/UI/Build EquipPanel Prefab
/// </summary>
public static class EquipPanelBuilder
{
    private const string Dir = "Assets/Resources/UI/Equip";
    private const string PrefabPath = Dir + "/EquipPanel.prefab";
    private const string ButtonPrefabPath = "Assets/Resources/UI/Common/CommonButton.prefab";
    private const string FontPath = "Assets/Art/Fonts/SIMHEI SDF.asset";
    private static TMP_FontAsset _font;
    private static GameObject _btnPrefab;
    [MenuItem("Tools/UI/Build EquipPanel Prefab")]
    public static void Build()
    {
        if (!Directory.Exists(Dir)) Directory.CreateDirectory(Dir);
        _font = AssetDatabase.LoadAssetAtPath<TMP_FontAsset>(FontPath);
        _btnPrefab = AssetDatabase.LoadAssetAtPath<GameObject>(ButtonPrefabPath);
        // ----- 根 -----
        var root = NewUI("EquipPanel", out var rootRt);
        Stretch(rootRt);
        root.AddComponent<Image>().color = new Color(0.10f, 0.11f, 0.14f, 1f);
        var cg = root.AddComponent<CanvasGroup>();

```

```

root.AddComponent<YOTOUIShow>();
// ----- 顶部:返回 / 资源金币 -----
var backBtn = BuildButton("Btn_Back", "返回", root.transform, 44);
var backRt = (RectTransform)backBtn.transform;
backRt.anchorMin = backRt.anchorMax = new Vector2(0, 1); backRt.pivot = new Vector2(0, 1);
backRt.anchoredPosition = new Vector2(40, -40); backRt.sizeDelta = new Vector2(220, 90);
// 资源(右上):金币 + 体力,均为「图标 + 数值」胶囊,不写文字
var coinText = UICurrencyPill.Build(root.transform, "Gold", UICurrencyPill.IconGold, _font,
    new Vector2(1, 1), new Vector2(-40, -40), new Vector2(300, 80));
var energyText = UICurrencyPill.Build(root.transform, "Energy", UICurrencyPill.IconEnergy, _font,
    new Vector2(1, 1), new Vector2(-40, -132), new Vector2(300, 80));
// ----- 标题 -----
var titleGo = NewUI("Title", out var titleRt, root.transform);
titleRt.anchorMin = new Vector2(0, 1); titleRt.anchorMax = new Vector2(1, 1); titleRt.pivot = new Vector2(0, 1);
titleRt.anchoredPosition = new Vector2(40, -180); titleRt.sizeDelta = new Vector2(-80, 90);
NewText(titleGo, "选择你的装备", 56, TextAlignmentOptions.Left);
// ----- 三个分类行 -----
var weaponRow = BuildSection("Weapon", "枪械", root.transform, -300);
var scopeRow = BuildSection("Scope", "瞄准镜", root.transform, -820);
var bulletRow = BuildSection("Bullet", "子弹", root.transform, -1340);
// ----- 出发 -----
var departBtn = BuildButton("Btn_Depart", "出发", root.transform, 56);
var departRt = (RectTransform)departBtn.transform;
departRt.anchorMin = departRt.anchorMax = new Vector2(0.5f, 0); departRt.pivot = new Vector2(0.5f, 0);
departRt.anchoredPosition = new Vector2(0, 80); departRt.sizeDelta = new Vector2(700, 200);
// ----- 接脚本字段 -----
var panel = root.AddComponent<EquipPanel>();
panel.canvasGroup = cg;
panel.uiType = UIEnum.EquipPanel;
panel.backBtn = backBtn;
panel.coinText = coinText;
panel.energyText = energyText;
panel.weaponRow = weaponRow;
panel.scopeRow = scopeRow;
panel.bulletRow = bulletRow;
panel.departBtn = departBtn;
PrefabUtility.SaveAsPrefabAsset(root, PrefabPath);
Object.DestroyImmediate(root);
AssetDatabase.SaveAssets();
AssetDatabase.Refresh();
Debug.Log($"[EquipPanelBuilder] Built {PrefabPath}");
}
// ===== 构件 =====
/// <summary>一个分类块:标签 + 横向滚动行。返回行内容容器(EquipPanel 往里铺卡)。</summary>
private static RectTransform BuildSection(string name, string label, Transform parent, float yTop)
{
    // 块:顶部对齐、左右铺满(留边),固定高度
    var block = NewUI($"Section_{name}", out var blockRt, parent);
    blockRt.anchorMin = new Vector2(0, 1); blockRt.anchorMax = new Vector2(1, 1); blockRt.pivot = new Vector2(0, 1);

```

```

r2(0.5f, 1);
    blockRt.anchoredPosition = new Vector2(0, yTop); blockRt.sizeDelta = new Vector2(-80, 480);
    // 标签
    var labelGo = NewUI("Label", out var labelRt, block.transform);
    labelRt.anchorMin = new Vector2(0, 1); labelRt.anchorMax = new Vector2(1, 1); labelRt.pivot = new Vector
r2(0, 1);
    labelRt.anchoredPosition = new Vector2(0, 0); labelRt.sizeDelta = new Vector2(0, 70);
    NewText(labelGo, label, 44, TextAlignmentOptions.Left);
    // 横向滚动行(标签下方)
    var scrollGo = NewUI("Scroll", out var scrollRt, block.transform);
    scrollRt.anchorMin = new Vector2(0, 0); scrollRt.anchorMax = new Vector2(1, 1); scrollRt.pivot = new Vector
ctor2(0.5f, 1);
    scrollRt.offsetMin = new Vector2(0, 0); scrollRt.offsetMax = new Vector2(0, -80); // 顶部让出标签
    var scroll = scrollGo.AddComponent<ScrollRect>();
    scroll.horizontal = true; scroll.vertical = false;
    scroll.movementType = ScrollRect.MovementType.Clamped;
    var viewportGo = NewUI("Viewport", out var viewportRt, scrollGo.transform);
    Stretch(viewportRt);
    viewportGo.AddComponent<Image>().color = new Color(1, 1, 1, 0.03f);
    viewportGo.AddComponent<RectMask2D>();
    var contentGo = NewUI("Content", out var contentRt, viewportGo.transform);
    contentRt.anchorMin = new Vector2(0, 0); contentRt.anchorMax = new Vector2(0, 1); contentRt.pivot = new Vector2(0, 0.5f);
    contentRt.anchoredPosition = Vector2.zero;
    var hlg = contentGo.AddComponent<HorizontalLayoutGroup>();
    hlg.spacing = 24;
    hlg.padding = new RectOffset(16, 16, 16, 16);
    hlg.childControlWidth = true; hlg.childControlHeight = true;
    hlg.childForceExpandWidth = false; hlg.childForceExpandHeight = false;
    hlg.childAlignment = TextAnchor.MiddleLeft;
    var fitter = contentGo.AddComponent<ContentSizeFitter>();
    fitter.horizontalFit = ContentSizeFitter.FitMode.PreferredSize;
    scroll.viewport = viewportRt; scroll.content = contentRt;
    return contentRt;
}
private static Button BuildButton(string name, string label, Transform parent, float fontSize)
{
    var go = (GameObject)PrefabUtility.InstantiatePrefab(_btnPrefab, parent);
    go.name = name;
    go.SetActive(true);
    var text = go.GetComponentInChildren<TextMeshProUGUI>(true);
    if (text != null) { text.text = label; text.fontSize = UITheme.Font(fontSize); }
    return go.GetComponent<Button>();
}
// ===== 工具 =====
private static GameObject NewUI(string name, out RectTransform rt, Transform parent = null)
{
    var go = new GameObject(name, typeof(RectTransform));
    rt = go.GetComponent<RectTransform>();
    if (parent != null) rt.SetParent(parent, false);

```

```

    return go;
}
private static void Stretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero; rt.anchorMax = Vector2.one;
    rt.offsetMin = Vector2.zero; rt.offsetMax = Vector2.zero;
}
private static TextMeshProUGUI NewText(GameObject go, string text, float size, TextAlignmentOptions align)
{
    var tmp = go.AddComponent<TextMeshProUGUI>();
    tmp.text = text; tmp.fontSize = UITheme.Font(size); tmp.alignment = align; tmp.color = Color.white; tmp
    .raycastTarget = false;
    var font = _font != null ? _font : TMP_Settings.defaultFontAsset;
    if (font != null) tmp.font = font;
    return tmp;
}
}
#endif
#if UNITY_EDITOR
using System.IO;
using TMPro;
using UnityEditor;
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// 一键生成打猎结算界面预制体 FinishPanel.prefab 到 Resources/UI/Main 下(覆盖旧占位)。
/// 外壳:半透明遮罩(挡住身后 HUD/场景) + 居中卡片(标题 + 明细滚动列表 + 总积分 + 「返回大厅」)。
/// 逐种击杀明细由 <see cref="FinishPanel"/> 运行时按传入的 <c>HuntResult</c> 构建。
///
/// 尺寸按"画布宽恒为 1920 单位"(CanvasScaler match=width)给。
/// 菜单:Tools/UI/Build FinishPanel Prefab
/// </summary>
public static class FinishPanelBuilder
{
    private const string Dir = "Assets/Resources/UI/Main";
    private const string PrefabPath = Dir + "/FinishPanel.prefab";
    private const string ButtonPrefabPath = "Assets/Resources/UI/Common/CommonButton.prefab";
    private const string FontPath = "Assets/Art/Fonts/SIMHEI SDF.asset";
    private static TMP_FontAsset _font;
    private static GameObject _btnPrefab;
    [MenuItem("Tools/UI/Build FinishPanel Prefab")]
    public static void Build()
    {
        if (!Directory.Exists(Dir)) Directory.CreateDirectory(Dir);
        _font = AssetDatabase.LoadAssetAtPath<TMP_FontAsset>(FontPath);
        _btnPrefab = AssetDatabase.LoadAssetAtPath<GameObject>(ButtonPrefabPath);
        if (_btnPrefab == null)
        {
            Debug.LogError($"[FinishPanelBuilder] 未找到通用按钮 {ButtonPrefabPath},无法生成。");
            return;

```

```

}
// ----- 根(半透明遮罩,挡住身后并拦截点击)-----
var root = NewUI("FinishPanel", out var rootRt);
Stretch(rootRt);
root.AddComponent<Image>().color = new Color(0f, 0f, 0f, 0.6f);
var cg = root.AddComponent<CanvasGroup>();
root.AddComponent<YOTOUIShow>();
// ----- 居中卡片 -----
var card = NewUI("Card", out var cardRt, root.transform);
cardRt.anchorMin = cardRt.anchorMax = cardRt.pivot = new Vector2(0.5f, 0.5f);
cardRt.sizeDelta = new Vector2(1600, 2100); // 放大结算卡片(运行时 FinishPanel 也会同步成这个尺寸)
var cardBg = card.AddComponent<Image>();
cardBg.sprite = BuiltinSprite("UI/Skin/UISprite.psd");
cardBg.type = Image.Type.Sliced;
cardBg.color = new Color(0.94f, 0.93f, 0.97f, 1f);
// ----- 标题(卡片顶部)-----
var titleGo = NewUI("Title", out var titleRt, card.transform);
titleRt.anchorMin = new Vector2(0, 1); titleRt.anchorMax = new Vector2(1, 1); titleRt.pivot = new Vector2(0.5f, 1);
titleRt.anchoredPosition = new Vector2(0, -50); titleRt.sizeDelta = new Vector2(0, 140);
var titleText = NewText(titleGo, "打猎结算", 76, TextAlignmentOptions.Center, new Color(0.25f, 0.24f, 0.30f, 1f));
// ----- 明细滚动列表 -----
var scrollGo = NewUI("Scroll", out var scrollRt, card.transform);
scrollRt.anchorMin = new Vector2(0, 0); scrollRt.anchorMax = new Vector2(1, 1);
scrollRt.offsetMin = new Vector2(60, 340); scrollRt.offsetMax = new Vector2(-60, -230);
var scroll = scrollGo.AddComponent<ScrollRect>();
scroll.horizontal = false; scroll.vertical = true; scroll.scrollSensitivity = 40f;
scroll.movementType = ScrollRect.MovementType.Clamped;
var viewport = NewUI("Viewport", out var viewportRt, scrollGo.transform);
Stretch(viewportRt);
var vplmg = viewport.AddComponent<Image>();
vplmg.sprite = BuiltinSprite("UI/Skin/UISprite.psd");
vplmg.type = Image.Type.Sliced;
vplmg.color = new Color(0.86f, 0.84f, 0.93f, 1f);
viewport.AddComponent<RectMask2D>();
var content = NewUI("Content", out var contentRt, viewport.transform);
contentRt.anchorMin = new Vector2(0, 1); contentRt.anchorMax = new Vector2(1, 1); contentRt.pivot = new Vector2(0.5f, 1);
contentRt.anchoredPosition = Vector2.zero; contentRt.sizeDelta = Vector2.zero;
var vlg = content.AddComponent<VerticalLayoutGroup>();
vlg.spacing = 16;
vlg.padding = new RectOffset(20, 20, 20, 20);
vlg.childAlignment = TextAnchor.UpperCenter;
vlg.childControlWidth = true; vlg.childForceExpandWidth = true;
vlg.childControlHeight = true; vlg.childForceExpandHeight = false;
var csf = content.AddComponent<ContentSizeFitter>();
csf.verticalFit = ContentSizeFitter.FitMode.PreferredSize;
scroll.viewport = viewportRt;
scroll.content = contentRt;

```

```

// ----- 总积分(列表下方)-----
var totalGo = NewUI("Total", out var totalRt, card.transform);
totalRt.anchorMin = new Vector2(0, 0); totalRt.anchorMax = new Vector2(1, 0); totalRt.pivot = new Vector2(0.5f, 0);
totalRt.anchoredPosition = new Vector2(0, 190); totalRt.sizeDelta = new Vector2(-100, 100);
var totalText = NewText(totalGo, "总积分: 0", 52, TextAlignmentOptions.Right, new Color(0.25f, 0.24f, 0.30f, 1f));
// ----- 返回大厅按钮(卡片底部)-----
var confirmBtn = BuildButton("Btn_Confirm", "返回大厅", card.transform, 50);
var confirmRt = (RectTransform)confirmBtn.transform;
confirmRt.anchorMin = confirmRt.anchorMax = new Vector2(0.5f, 0); confirmRt.pivot = new Vector2(0.5f, 0);
confirmRt.anchoredPosition = new Vector2(0, 50); confirmRt.sizeDelta = new Vector2(420, 120);
// ----- 接脚本字段 -----
var panel = root.AddComponent<FinishPanel>();
panel.canvasGroup = cg;
panel.uiType = UIEnum.FinishPanel;
panel.titleText = titleText;
panel.totalText = totalText;
panel.content = contentRt;
panel.confirmBtn = confirmBtn;
PrefabUtility.SaveAsPrefabAsset(root, PrefabPath);
Object.DestroyImmediate(root);
AssetDatabase.SaveAssets();
AssetDatabase.Refresh();
Debug.Log($"[FinishPanelBuilder] Built {PrefabPath}");
}
// ===== 工具 =====
private static Button BuildButton(string name, string label, Transform parent, float fontSize)
{
    var go = (GameObject)PrefabUtility.InstantiatePrefab(_btnPrefab, parent);
    go.name = name;
    go.SetActive(true);
    var text = go.GetComponentInChildren<TextMeshProUGUI>(true);
    if (text != null) { text.text = label; text.fontSize = UITheme.Font(fontSize); }
    return go.GetComponent<Button>();
}
private static GameObject NewUI(string name, out RectTransform rt, Transform parent = null)
{
    var go = new GameObject(name, typeof(RectTransform));
    rt = go.GetComponent<RectTransform>();
    if (parent != null) rt.SetParent(parent, false);
    return go;
}
private static void Stretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero; rt.anchorMax = Vector2.one;
    rt.offsetMin = Vector2.zero; rt.offsetMax = Vector2.zero;
}
private static TextMeshProUGUI NewText(GameObject go, string text, float size, TextAlignmentOptions align,

```

```

private bool built;
/// <summary>子类把控件加到这里(滚动内容容器),而非直接加到自身。</summary>
protected RectTransform Content { get; private set; }
protected T Resolve<T>() where T : class
    => GameLoop.Instance != null && GameLoop.Instance.Ctx != null ? GameLoop.Instance.Ctx.Get<T>() : null;
protected virtual void OnEnable()
{
    if (!built)
    {
        SetupRoot();
        Build();
        built = true;
    }
    Refresh();
    Bind();
}
protected virtual void OnDisable() => Unbind();
/// <summary>构建本页签控件(只调一次)。</summary>
protected abstract void Build();
/// <summary>从服务拉最新值刷新显示(每次激活调用)。</summary>
protected virtual void Refresh() { }
protected virtual void Bind() { }
protected virtual void Unbind() { }
// ===== 运行时 UI 工具 =====
private void SetupRoot()
{
    // 容器做成 ScrollRect:内容超出可视区(如按键 8 行)时纵向滚动,不会向下溢出压到返回按钮。
    var scroll = gameObject.AddComponent<ScrollRect>();
    scroll.horizontal = false;
    scroll.vertical = true;
    scroll.movementType = ScrollRect.MovementType.Clamped;
    scroll.scrollSensitivity = 30f;
    var viewport = NewUI("Viewport", transform);
    var vpRt = (RectTransform)viewport.transform;
    vpRt.anchorMin = Vector2.zero; vpRt.anchorMax = Vector2.one; vpRt.offsetMin = Vector2.zero; vpRt.offset
Max = Vector2.zero;
    viewport.AddComponent<RectMask2D>();
    viewport.AddComponent<Image>().color = new Color(0f, 0f, 0f, 0.001f); // 透明底,空白处也能接拖拽滚动
    var contentGo = NewUI("Content", viewport.transform);
    var contentRt = (RectTransform)contentGo.transform;
    contentRt.anchorMin = new Vector2(0, 1); contentRt.anchorMax = new Vector2(1, 1); contentRt.pivot = new
Vector2(0.5f, 1);
    contentRt.anchoredPosition = Vector2.zero; contentRt.sizeDelta = Vector2.zero;
    var vlg = contentGo.AddComponent<VerticalLayoutGroup>();
    vlg.spacing = 24;
    vlg.padding = new RectOffset(40, 40, 32, 32);
    vlg.childAlignment = TextAnchor.UpperCenter;
    vlg.childControlWidth = true;
    vlg.childControlHeight = true;
    vlg.childForceExpandWidth = true;

```

```

    vlg.childForceExpandHeight = false;
    var fitter = contentGo.AddComponent<ContentSizeFitter>();
    fitter.verticalFit = ContentSizeFitter.FitMode.PreferredSize;
    scroll.viewport = vpRt;
    scroll.content = contentRt;
    Content = contentRt;
}

protected static GameObject NewUI(string name, Transform parent)
{
    var go = new GameObject(name, typeof(RectTransform));
    go.transform.SetParent(parent, false);
    return go;
}

protected TextMeshProUGUI NewText(Transform parent, string text, float size, TextAlignmentOptions align)
{
    var go = NewUI("Text", parent);
    var tmp = go.AddComponent<TextMeshProUGUI>();
    tmp.text = text;
    tmp.fontSize = UITheme.Font(size);
    tmp.alignment = align;
    tmp.color = Color.white;
    tmp.raycastTarget = false;
    var f = font != null ? font : TMP_Settings.defaultFontAsset;
    if (f != null) tmp.font = f;
    return tmp;
}

/// <summary>一行(水平布局),高度固定。</summary>
protected GameObject NewRow(Transform parent, float height = 100)
{
    var go = NewUI("Row", parent);
    var hlg = go.AddComponent<HorizontalLayoutGroup>();
    hlg.spacing = 12;
    hlg.padding = new RectOffset(8, 8, 0, 0);
    hlg.childAlignment = TextAnchor.MiddleLeft;
    hlg.childControlWidth = true;
    hlg.childControlHeight = true;
    hlg.childForceExpandWidth = false;
    hlg.childForceExpandHeight = false;
    var le = go.AddComponent<LayoutElement>();
    le.minHeight = height;
    le.preferredHeight = height;
    return go;
}

protected TextMeshProUGUI NewLabel(Transform parent, string text, float width, float size, TextAlignmentOptions align)
{
    var t = NewText(parent, text, size, align);
    var le = t.gameObject.AddComponent<LayoutElement>();
    le.minWidth = width;
    le.preferredWidth = width;
}

```



```

    return t;
}
protected Image NewImage(Transform parent, string name, Color color)
{
    var go = NewUI(name, parent);
    var img = go.AddComponent<Image>();
    img.color = color;
    return img;
}
protected Button NewButton(Transform parent, string label, float width, float height, float fontSize = 22)
{
    var go = NewUI("Button", parent);
    var le = go.AddComponent<LayoutElement>();
    le.minWidth = width; le.preferredWidth = width;
    le.minHeight = height; le.preferredHeight = height;
    var img = go.AddComponent<Image>();
    img.color = new Color(0.3f, 0.32f, 0.4f, 1f);
    var btn = go.AddComponent<Button>();
    btn.targetGraphic = img;
    var lbl = NewText(go.transform, label, fontSize, TextAlignmentOptions.Center);
    Stretch(lbl.rectTransform);
    return btn;
}
protected Slider NewSlider(Transform parent)
{
    var go = NewUI("Slider", parent);
    var le = go.AddComponent<LayoutElement>();
    le.flexibleWidth = 1; le.minWidth = 360; le.preferredHeight = 52;
    var slider = go.AddComponent<Slider>();
    var bg = NewImage(go.transform, "Background", new Color(0.3f, 0.3f, 0.35f, 1f));
    var bgRt = bg.rectTransform; bgRt.anchorMin = new Vector2(0, 0.3f); bgRt.anchorMax = new Vector2(1, 0.7
f); bgRt.offsetMin = Vector2.zero; bgRt.offsetMax = Vector2.zero;
    var fillArea = NewUI("Fill Area", go.transform); var faRt = (RectTransform)fillArea.transform;
    faRt.anchorMin = new Vector2(0, 0.25f); faRt.anchorMax = new Vector2(1, 0.75f); faRt.offsetMin = new Ve
ctor2(10, 0); faRt.offsetMax = new Vector2(-10, 0);
    var fill = NewImage(fillArea.transform, "Fill", new Color(0.4f, 0.6f, 0.9f, 1f));
    var fillRt = fill.rectTransform; fillRt.anchorMin = new Vector2(0, 0); fillRt.anchorMax = new Vector2(0
, 1); fillRt.sizeDelta = new Vector2(18, 0);
    var handleArea = NewUI("Handle Slide Area", go.transform); var haRt = (RectTransform)handleArea.transfo
rm;
    haRt.anchorMin = Vector2.zero; haRt.anchorMax = Vector2.one; haRt.offsetMin = new Vector2(10, 0); haRt.
offsetMax = new Vector2(-10, 0);
    var handle = NewImage(handleArea.transform, "Handle", Color.white);
    var handleRt = handle.rectTransform; handleRt.sizeDelta = new Vector2(40, 0);
    slider.fillRect = fillRt;
    slider.handleRect = handleRt;
    slider.targetGraphic = handle;
    slider.direction = Slider.Direction.LeftToRight;
    slider.minValue = 0; slider.maxValue = 1; slider.value = 1;
    return slider;
}

```

```

}
protected Toggle NewToggle(Transform parent)
{
    var go = NewUI("Toggle", parent);
    var le = go.AddComponent<LayoutElement>();
    le.minWidth = 64; le.preferredWidth = 64; le.preferredHeight = 64;
    var toggle = go.AddComponent<Toggle>();
    var bg = NewImage(go.transform, "Background", new Color(0.25f, 0.27f, 0.32f, 1f));
    var bgRt = bg.rectTransform; bgRt.anchorMin = new Vector2(0, 0.5f); bgRt.anchorMax = new Vector2(0, 0.5f); bgRt.pivot = new Vector2(0, 0.5f);
    bgRt.sizeDelta = new Vector2(56, 56); bgRt.anchoredPosition = Vector2.zero;
    var check = NewImage(bg.transform, "Checkmark", new Color(0.4f, 0.85f, 0.45f, 1f));
    var cRt = check.rectTransform; cRt.anchorMin = new Vector2(0.5f, 0.5f); cRt.anchorMax = new Vector2(0.5f, 0.5f); cRt.sizeDelta = new Vector2(36, 36); cRt.anchoredPosition = Vector2.zero;
    toggle.targetGraphic = bg;
    toggle.graphic = check;
    return toggle;
}
protected static void Stretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero; rt.anchorMax = Vector2.one;
    rt.offsetMin = Vector2.zero; rt.offsetMax = Vector2.zero;
}
}
using TMPro;
using UnityEngine;
using UnityEngine.UI;

/// <summary>声音设置页签:音乐/音效 两路音量滑条 + 静音开关,绑定 SoundMgr(改动即生效并存盘)。</summary>
public class SoundSettingsTab : SettingTabBase
{
    private static readonly SoundChannel[] Channels = { SoundChannel.Music, SoundChannel.Sfx };
    private static readonly string[] Names = { "音乐", "音效" };
    private readonly Slider[] sliders = new Slider[Channels.Length];
    private readonly TextMeshProUGUI[] values = new TextMeshProUGUI[Channels.Length];
    private readonly Toggle[] mutes = new Toggle[Channels.Length];
    private bool refreshing;
    protected override void Build()
    {
        NewLabel(Content, "声音设置", 700, 52, TextAlignmentOptions.Left);
        for (int i = 0; i < Channels.Length; i++)
        {
            int idx = i;
            var row = NewRow(Content);
            NewLabel(row.transform, Names[i], 220, 40, TextAlignmentOptions.Left);
            sliders[i] = NewSlider(row.transform);
            values[i] = NewLabel(row.transform, "100%", 130, 36, TextAlignmentOptions.Right);
            NewLabel(row.transform, "静音", 110, 34, TextAlignmentOptions.Right);
            mutes[i] = NewToggle(row.transform);
            sliders[i].onValueChanged.AddListener(v => OnVolume(idx, v));
            mutes[i].onValueChanged.AddListener(b => OnMute(idx, b));
        }
    }
}

```

```

    }
}
protected override void Refresh()
{
    var sound = Resolve<SoundMgr>();
    if (sound == null) return;
    refreshing = true;
    for (int i = 0; i < Channels.Length; i++)
    {
        float v = ChannelVolume(sound, i);
        sliders[i].SetValueWithoutNotify(v);
        values[i].text = Pct(v);
        mutes[i].SetIsOnWithoutNotify(sound.IsChannelMuted(Channels[i]));
    }
    refreshing = false;
}
private void OnVolume(int idx, float v)
{
    if (refreshing) return;
    var sound = Resolve<SoundMgr>();
    if (sound == null) return;
    switch (Channels[idx])
    {
        case SoundChannel.Master: sound.SetMasterVolume(v); break;
        case SoundChannel.Music: sound.SetBgmVolume(v); break;
        case SoundChannel.Sfx: sound.SetSfxVolume(v); break;
        case SoundChannel.Ui: sound.SetUiVolume(v); break;
    }
    values[idx].text = Pct(v);
}
private void OnMute(int idx, bool muted)
{
    if (refreshing) return;
    var sound = Resolve<SoundMgr>();
    if (sound == null) return;
    switch (Channels[idx])
    {
        case SoundChannel.Master: sound.SetMasterMuted(muted); break;
        case SoundChannel.Music: sound.SetMusicMuted(muted); break;
        case SoundChannel.Sfx: sound.SetSfxMuted(muted); break;
        case SoundChannel.Ui: sound.SetUiMuted(muted); break;
    }
}
private static float ChannelVolume(SoundMgr sound, int idx)
{
    switch (Channels[idx])
    {
        case SoundChannel.Master: return sound.MasterVolume;
        case SoundChannel.Music: return sound.MusicVolume;
        case SoundChannel.Sfx: return sound.SfxVolume;
    }
}

```

```

        default: return sound.UiVolume;
    }
}

private static string Pct(float v) => $"{Mathf.RoundToInt(v * 100f)}%";
}

using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.UI;
using YFramework.Config;
using YOTO;

/// <summary>
/// 商店面板(<see cref="UIEnum.ShopPanel"/>):分类页签 + 3 列卡片网格。
/// 顶部:返回(关闭) / 资源金币 / 商人头像
/// 中部:当前分类的卡片网格(图片 + 名称 + 价格 + 购买按钮:点击购买弹确认页,确认后购买,结果用飘字反馈)
/// 底部:武器 / 瞄准镜 / 子弹 三个分类页签
/// 目录与撮合在 <see cref="ShopSystem"/>(按 <see cref="ShopCategory"/> 过滤);卡片运行时按目录构建。
/// 余额变化(RefreshCurrency)时刷新金币与各卡买得起/买不起。预制体外壳由 <c>Tools/UI/Build ShopPanel Prefab</c>
    生成。
/// </summary>
public class ShopPanel : UIPageBase
{
    [Header("顶部")]
    public Button backBtn;
    public TextMeshProUGUI coinText; // 金币数值(图标在预制体胶囊里)
    public TextMeshProUGUI energyText; // 体力数值(图标在预制体胶囊里)
    public Image merchantAvatar;
    [Header("网格")]
    public RectTransform grid; // GridLayoutGroup 容器(滚动视图内)
    [Header("底部页签")]
    public Button tabWeapon;
    public Button tabScope;
    public Button tabBullet;
    private static readonly Color Affordable = new Color(0.30f, 0.78f, 0.36f, 1f); // 绿:买得起
    private static readonly Color Unaffordable = new Color(0.32f, 0.34f, 0.40f, 1f); // 灰:买不起
    private static readonly Color TabOn = new Color(0.30f, 0.55f, 0.85f, 1f);
    private static readonly Color TabOff = new Color(0.25f, 0.27f, 0.33f, 1f);
    private ShopSystem shop;
    private CurrencySystem currency;
    private LoadoutSystem loadout; // 判断装备是否已拥有(已拥有的按钮置灰)
    private ResMgr resMgr;
    private EventMgr eventMgr;
    private TMP_FontAsset font;
    private ShopCategory current = ShopCategory.Weapon;
    private WeaponModelPreview modelPreview; // 模型转台(占原商人头像位,扩大到约 2/5 屏);三类(枪/镜/弹)点卡切
    换展示
    private int selectedId; // 当前展示/选中的物品 id(随分类切换)
    private readonly List<(int id, GameObject card)> previewCards = new(); // 当前分类带模型的卡(就地切换描边,
    避免点击时重建销毁自身)
    public override void OnLoad()

```

```

{
    shop = GetService<ShopSystem>();
    currency = GetService<CurrencySystem>();
    loadout = GetService<LoadoutSystem>();
    resMgr = GetService<ResMgr>();
    eventMgr = GetService<EventMgr>();
    if (coinText != null) font = coinText.font; // 复用外壳的中文字体给运行时卡片
    if (backBtn != null) backBtn.onClick.AddListener(CloseSelf);
    if (tabWeapon != null) tabWeapon.onClick.AddListener(() => SelectCategory(ShopCategory.Weapon));
    if (tabScope != null) tabScope.onClick.AddListener(() => SelectCategory(ShopCategory.Scope));
    if (tabBullet != null) tabBullet.onClick.AddListener(() => SelectCategory(ShopCategory.Bullet));
    modelPreview = new WeaponModelPreview(CreatePreviewHost(), resMgr);
}

/// <summary>模型展示框:占原「商人头像」位并扩大到约屏幕上方 2/5;同时把下方网格下压让出空间。</summary>
private RectTransform CreatePreviewHost()
{
    // 网格(Scroll)下压:只占下半屏(顶部让给展示区,底部留页签)
    if (grid != null && grid.parent != null && grid.parent.parent is RectTransform scroll)
    {
        scroll.anchorMin = new Vector2(0f, 0f);
        scroll.anchorMax = new Vector2(1f, 0.48f);
        scroll.offsetMin = new Vector2(30f, 240f);
        scroll.offsetMax = new Vector2(-30f, 0f);
    }
    if (merchantAvatar != null)
    {
        merchantAvatar.enabled = false; // 让位给模型展示
        var ar = (RectTransform)merchantAvatar.transform;
        ar.anchorMin = new Vector2(0.06f, 0.50f); // 上方 ~2/5 屏(50%~93% 高度)
        ar.anchorMax = new Vector2(0.94f, 0.93f);
        ar.offsetMin = Vector2.zero; ar.offsetMax = Vector2.zero;
        var go = new GameObject("ModelPreview", typeof(RectTransform));
        var rt = (RectTransform)go.transform;
        rt.SetParent(merchantAvatar.transform, false);
        rt.anchorMin = Vector2.zero; rt.anchorMax = Vector2.one;
        rt.offsetMin = Vector2.zero; rt.offsetMax = Vector2.zero; // 填满展示区
        return rt;
    }
    // 兜底:顶部一大块
    var host = new GameObject("ModelPreview", typeof(RectTransform));
    host.transform.SetParent(transform, false);
    var hr = (RectTransform)host.transform;
    hr.anchorMin = new Vector2(0.06f, 0.50f);
    hr.anchorMax = new Vector2(0.94f, 0.93f);
    hr.offsetMin = Vector2.zero; hr.offsetMax = Vector2.zero;
    return hr;
}

public override void OnShow()
{
    eventMgr?.Add(YOTOEventType.RefreshCurrency, OnCurrencyChanged);
}

```

```

eventMgr?.Add(YOTOEventType.RefreshLoadout, OnLoadoutChanged); // 购买装备解锁后刷新「已拥有」灰按钮
SelectCategory(ShopCategory.Weapon);
RefreshCoin();
}
public override void OnHide()
{
    eventMgr?.Remove(YOTOEventType.RefreshCurrency, OnCurrencyChanged);
    eventMgr?.Remove(YOTOEventType.RefreshLoadout, OnLoadoutChanged);
    modelPreview?.SetActive(false);
}
public override void OnResize() { }
private void OnCurrencyChanged()
{
    RefreshCoin();
    RebuildGrid(); // 重建以刷新各卡绿/灰(买得起/买不起)
}
/// <summary>装备拥有变化(购买解锁):重建以把已拥有的按钮刷成灰「已拥有」。
/// 购买流程里 TrySpend 先触发 RefreshCurrency(此时尚未 Grant,仍显示「购买」),Grant 后的本事件才是正确态。<
/summary>
private void OnLoadoutChanged() => RebuildGrid();
// ----- 顶部 -----
private void RefreshCoin()
{
    if (currency == null) return;
    if (coinText != null) coinText.text = currency.Get(CurrencyType.Gold).ToString(); // 只填数值,图标
在胶囊里
    if (energyText != null) energyText.text = currency.Get(CurrencyType.Energy).ToString();
}
// ----- 分类页签 -----
private void SelectCategory(ShopCategory cat)
{
    current = cat;
    SetTabColor(tabWeapon, cat == ShopCategory.Weapon);
    SetTabColor(tabScope, cat == ShopCategory.Scope);
    SetTabColor(tabBullet, cat == ShopCategory.Bullet);
    EnsureDefaultSelection(); // 切到新分类:默认选第一件有模型的
    RebuildGrid();
    UpdatePreview();
}
/// <summary>当前分类未选中(或选中项已不在本类目录)时,默认选第一件有模型的。</summary>
private void EnsureDefaultSelection()
{
    var list = shop.CatalogOf(current);
    for (int i = 0; i < list.Count; i++)
        if ((int)list[i].Id == selectedId) return; // 当前选中仍在本类目录中
    selectedId = 0;
    for (int i = 0; i < list.Count; i++)
        if (!string.IsNullOrEmpty(list[i].ModelPath)) { selectedId = (int)list[i].Id; break; }
}
/// <summary>点卡片:选中它并在上方展示其模型。**就地**切换描边——不重建网格,

```

```

/// 否则会在卡片自身的 onClick 里把自己 Destroy 掉,破坏 EventSystem(点几下后点击失灵/描边卡死)。</summary>
private void SelectPreview(int id)
{
    selectedId = id;
    for (int i = 0; i < previewCards.Count; i++)
        ApplyOutline(previewCards[i].card, previewCards[i].id == id);
    UpdatePreview();
}

private const string SelectFrameName = "SelectFrame";
/// <summary>给卡片加/去金色选中边框(就地,不销毁卡片)。
/// 用 4 条细边拼出"框",而不是 Outline 组件——后者在本项目渲染下会糊成一整片金色底图,达不到"金色框"效果。</s
ummary>
private static void ApplyOutline(GameObject card, bool on)
{
    if (card == null) return;
    var existing = card.transform.Find(SelectFrameName);
    if (!on)
    {
        if (existing != null) Destroy(existing.gameObject);
        return;
    }
    if (existing != null) return; // 已有边框
    var frameGo = new GameObject(SelectFrameName, typeof(RectTransform));
    var frame = (RectTransform)frameGo.transform;
    frame.SetParent(card.transform, false);
    frame.anchorMin = Vector2.zero; frame.anchorMax = Vector2.one;
    frame.offsetMin = Vector2.zero; frame.offsetMax = Vector2.zero;
    frame.SetAsLastSibling(); // 边框画在卡片内容之上
    const float t = 8f; // 边宽
    var gold = new Color(1f, 0.85f, 0.2f, 1f);
    AddEdge(frame, new Vector2(0, 1), new Vector2(1, 1), new Vector2(0, -t), new Vector2(0, 0), gold); //
上
    AddEdge(frame, new Vector2(0, 0), new Vector2(1, 0), new Vector2(0, 0), new Vector2(0, t), gold); //
下
    AddEdge(frame, new Vector2(0, 0), new Vector2(0, 1), new Vector2(0, 0), new Vector2(t, 0), gold); //
左
    AddEdge(frame, new Vector2(1, 0), new Vector2(1, 1), new Vector2(-t, 0), new Vector2(0, 0), gold); //
右
}
/// <summary>在 frame 下加一条贴边的纯色 Image(选中框的一条边)。</summary>
private static void AddEdge(RectTransform parent, Vector2 aMin, Vector2 aMax, Vector2 offMin, Vector2 offMa
x, Color color)
{
    var go = new GameObject("Edge", typeof(RectTransform), typeof(Image));
    var rt = (RectTransform)go.transform;
    rt.SetParent(parent, false);
    rt.anchorMin = aMin; rt.anchorMax = aMax;
    rt.offsetMin = offMin; rt.offsetMax = offMax;
    var img = go.GetComponent<Image>();
    img.color = color;
}

```

```

img.raycastTarget = false;
}
/// <summary>上方转台展示当前分类选中物品的模型(三类通用:枪/镜/弹)。</summary>
private void UpdatePreview()
{
    if (modelPreview == null) return;
    string path = null;
    if (selectedId > 0)
    {
        var list = shop.CatalogOf(current);
        for (int i = 0; i < list.Count; i++)
            if ((int)list[i].Id == selectedId) { path = list[i].ModelPath; break; }
    }
    if (string.IsNullOrEmpty(path)) { modelPreview.SetActive(false); return; }
    modelPreview.SetActive(true);
    modelPreview.Show(path);
}
private static void SetTabColor(Button btn, bool on)
{
    if (btn == null) return;
    var img = btn.targetGraphic as Image;
    if (img != null) img.color = on ? TabOn : TabOff;
}
// ----- 卡片网格 -----
private void RebuildGrid()
{
    if (grid == null) return;
    previewCards.Clear();
    for (int i = grid.childCount - 1; i >= 0; i--)
    {
        var c = grid.GetChild(i);
        c.SetParent(null, false); // 先脱离父级再销毁:避免本帧旧卡与新卡并存导致布局/计数错乱
        Destroy(c.gameObject);
    }
    var list = shop.CatalogOf(current);
    for (int i = 0; i < list.Count; i++) BuildCard(list[i]);
}
private void BuildCard(Item item)
{
    // 卡片容器:中性卡底(不再整卡点击购买;改为卡内「购买」按钮 → 确认弹窗 → 购买)。
    // 各区块按 cell(550×560)从下往上排,文本带高放大到能容下 ×2 字号(否则名称/价格会被裁掉看不见)。
    var card = new GameObject($"Card_{item.Id}", typeof(RectTransform), typeof(Image));
    card.transform.SetParent(grid, false); // 尺寸由 GridLayoutGroup 决定
    var bg = card.GetComponent<Image>();
    bg.color = new Color(0.18f, 0.20f, 0.25f, 1f);
    bg.raycastTarget = false;
    // 任意带模型的卡(枪/镜/弹):整卡可点 = 选中它并在上方展示其模型(底部「购买」按钮在上层,各点各的)
    if (!string.IsNullOrEmpty(item.ModelPath))
    {
        bg.raycastTarget = true;
    }
}

```



```

var cardBtn = card.AddComponent<Button>();
cardBtn.transition = Selectable.Transition.None; // 不改卡底色
cardBtn.targetGraphic = bg;
int sid = (int)item.Id;
cardBtn.onClick.AddListener(() => SelectPreview(sid));
previewCards.Add((sid, card));
ApplyOutline(card, sid == selectedId); // 当前选中:金色边框
}

// 图片(上部,占大半:y 270 → 顶):与装备页一致,用渲染出的 3D 道具侧视快照(全局共享缓存,跨面板复用已渲图),
// 无模型才回退 2D 图标。统一走 Image + preserveAspect:正方形快照按比例居中,不被卡片图框拉伸。
var pic = NewChild(card.transform, "Pic", out var picRt);
picRt.anchorMin = Vector2.zero; picRt.anchorMax = Vector2.one;
picRt.offsetMin = new Vector2(20, 270); picRt.offsetMax = new Vector2(-20, -20);
var picImg = pic.AddComponent<Image>();
picImg.raycastTarget = false; picImg.preserveAspect = true;
var sprite = ModelSnapshotCache.CardSprite(item.ModelPath, resMgr) // 全局共享缓存复用的侧视
快照
    ?? (!string.IsNullOrEmpty(item.IconPath) ? resMgr.Load<Sprite>(item.IconPath) : null); //
无模型回退 2D 图标
picImg.sprite = sprite; picImg.enabled = sprite != null;
// 名称(y 196..266,带高 70 容下 30pt×2)
var name = NewChild(card.transform, "Name", out var nameRt);
nameRt.anchorMin = new Vector2(0, 0); nameRt.anchorMax = new Vector2(1, 0); nameRt.pivot = new Vector2(
0.5f, 0);
nameRt.offsetMin = new Vector2(6, 196); nameRt.offsetMax = new Vector2(-6, 266);
NewText(name, item.Name, 30, TextAlignmentOptions.Center, Color.white);
// 价格(y 130..190,带高 60 容下 26pt×2,带货币色)
var price = NewChild(card.transform, "Price", out var priceRt);
priceRt.anchorMin = new Vector2(0, 0); priceRt.anchorMax = new Vector2(1, 0); priceRt.pivot = new Vector2(
0.5f, 0);
priceRt.offsetMin = new Vector2(6, 130); priceRt.offsetMax = new Vector2(-6, 190);
string priceDesc = $"{currency.DisplayName((CurrencyType)item.PriceType)} {item.Price}";
NewText(price, priceDesc, 26, TextAlignmentOptions.Center, new Color(1f, 0.83f, 0.47f, 1f));
// 购买按钮(底部 y 20..115):已拥有=灰「已拥有」/ 买得起=绿 / 买不起=灰。始终可点——给对应飘字反馈。
bool owned = loadout != null && item.ShopCategory != 0 && loadout.IsOwned((int)item.Id);
bool affordable = !owned && shop.CanAfford(item);
var buyGo = NewChild(card.transform, "Buy", out var buyRt);
buyRt.anchorMin = new Vector2(0, 0); buyRt.anchorMax = new Vector2(1, 0); buyRt.pivot = new Vector2(0.5
f, 0);
buyRt.offsetMin = new Vector2(20, 20); buyRt.offsetMax = new Vector2(-20, 115);
var buyImg = buyGo.AddComponent<Image>();
buyImg.color = affordable ? Affordable : Unaffordable; // 已拥有/买不起都用灰
var buyBtn = buyGo.AddComponent<Button>();
buyBtn.targetGraphic = buyImg;
var label = NewChild(buyGo.transform, "Label", out var labelRt);
labelRt.anchorMin = Vector2.zero; labelRt.anchorMax = Vector2.one;
labelRt.offsetMin = Vector2.zero; labelRt.offsetMax = Vector2.zero;
NewText(label, owned ? "已拥有" : "购买", 28, TextAlignmentOptions.Center, Color.white);
int id = (int)item.Id; // 闭包捕获副本
string itemName = item.Name;

```

```

var priceType = (CurrencyType)item.PriceType;
if (owned)
    buyBtn.onClick.AddListener(() => FlyText($"{itemName}已拥有"));
else
    buyBtn.onClick.AddListener(() => OnBuyClick(id, itemName, priceDesc, priceType, affordable));
}
/// <summary>
/// 点击「购买」:买得起→弹确认页,确认后 <see cref="DoBuy"/> 购买;买不起→直接飘「xx 不足」,不弹确认。
/// </summary>
private void OnBuyClick(int id, string itemName, string priceDesc, CurrencyType priceType, bool affordable)
{
    if (!affordable)
    {
        FlyText($"{currency.DisplayName(priceType)}不足");
        return;
    }
    Show<ConfirmPanel, ConfirmParam>(new ConfirmParam
    {
        title = "购买确认",
        message = $"确定花费 {priceDesc} 购买\n「{itemName}」吗? ",
        confirmText = "购买",
        cancelText = "取消",
        onConfirm = () => DoBuy(id, itemName, priceType),
    });
}
/// <summary>真正购买并按结果飘字:成功「xx购买成功!」,失败按原因飘「金币不足 / 已购买 / 背包已满」。</summary>
private void DoBuy(int id, string itemName, CurrencyType priceType)
{
    switch (shop.Buy(id))
    {
        case BuyResult.Success: ShowReward("购买成功", RewardEntry.Item(id, 1)); break;
        case BuyResult.AlreadyOwned: FlyText($"{itemName}已购买"); break;
        case BuyResult.NotEnoughCurrency: FlyText($"{currency.DisplayName(priceType)}不足"); break;
        case BuyResult.BagFull: FlyText("背包已满"); break;
        default: FlyText("无法购买"); break;
    }
}
/// <summary>屏幕中央飘字提示(复用框架飘字系统)。</summary>
private void FlyText(string msg) => GetService<FlyTextMgr>()?.AddTextAtScreenCenter(msg);
/// <summary>通用奖励领取弹窗(Top 层,1 秒自动消失)。奖励已由 <see cref="ShopSystem"/> 入账,这里仅展示。</summary>
private void ShowReward(string title, params RewardEntry[] rewards)
    => Show<RewardClaimPanel, RewardClaimParam>(new RewardClaimParam
    {
        title = title,
        rewards = new System.Collections.Generic.List<RewardEntry>(rewards),
    });
// ----- 工具 -----
private GameObject NewChild(Transform parent, string name, out RectTransform rt)

```

```

{
    var go = new GameObject(name, typeof(RectTransform));
    rt = (RectTransform)go.transform;
    rt.SetParent(parent, false);
    return go;
}

private void NewText(GameObject go, string text, float size, TextAlignmentOptions align, Color color)
{
    var tmp = go.AddComponent<TextMeshProUGUI>();
    tmp.text = text;
    tmp.fontSize = UITheme.Font(size);
    tmp.alignment = align;
    tmp.color = color;
    tmp.raycastTarget = false;
    tmp.enableWordWrapping = false;
    tmp.overflowMode = TextOverflowModes.Ellipsis;
    var f = font != null ? font : TMP_Settings.defaultFontAsset;
    if (f != null) tmp.font = f;
}
}

using TMPro;
using UnityEngine;
using UnityEngine.UI;
using YOTO;

/// <summary>
/// 主界面(大厅):竖屏手机布局。
/// 左上:设置(方形按钮)    右上:体力 + 金币两项资源(<see cref="CurrencySystem"/>),体力后带「广告补充」按钮
/// 底部:左侧竖排 任务/商店 · 中间 准备(主按钮) · 右侧竖排 图鉴/排行榜
/// 预制体由 <c>Tools/UI/Build StartPanel Prefab</c> 程序化生成,脚本字段在那里接好。
///
/// 行为:设置→<see cref="SettingPanel"/>;商店→<see cref="ShopPanel"/>;任务→<see cref="TaskPanel"/>;图鉴→<see cref="CodexPanel"/>;排行榜→<see cref="LeaderboardPanel"/>;
/// 准备(中间主按钮)→选关 <see cref="MapSelectPanel"/>(选好关卡再进装备界面「出发」)。
/// 体力广告按钮→请求激励广告(<see cref="IAdService"/>,未接入则不发奖,见 <see cref="OnEnergyAdClick"/>)。
/// </summary>

public class StartPanel : UIPageBase
{
    [Header("顶部资源")]
    public TextMeshProUGUI energyText; // 体力
    public TextMeshProUGUI goldText; // 金币
    [Header("按钮")]
    public Button btn_setting;
    public Button btn_energyAd; // 体力后的「广告补充体力」按钮(预留接口)
    public Button btn_shop;
    public Button btn_task;
    public Button btn_prepare;
    public Button btn_codex;
    public Button btn_leaderboard;
    private const int EnergyAdReward = 1; // 看完一次激励广告补 1 点体力

```

```

private const string AdPlacementEnergy = "energy_refill"; // 体力广告位标识
private CurrencySystem currency;
private EventMgr eventMgr;
private DailyAdEnergySystem dailyAd; // 每日广告补体力次数(走 StoreMgr → 本地+云)
public override void OnLoad()
{
    currency = GetService<CurrencySystem>();
    eventMgr = GetService<EventMgr>();
    dailyAd = GetService<DailyAdEnergySystem>();
    if (btn_setting != null) btn_setting.onClick.AddListener(OnSettingClick);
    if (btn_energyAd != null) btn_energyAd.onClick.AddListener(OnEnergyAdClick);
    if (btn_shop != null) btn_shop.onClick.AddListener(OnShopClick);
    if (btn_prepare != null) btn_prepare.onClick.AddListener(OnPrepareClick);
    if (btn_codex != null) btn_codex.onClick.AddListener(OnCodexClick);
    if (btn_task != null) btn_task.onClick.AddListener(OnTaskClick);
    if (btn_leaderboard != null) btn_leaderboard.onClick.AddListener(OnLeaderboardClick);
}
public override void OnShow()
{
    eventMgr?.Add(YOTOEventType.RefreshCurrency, RefreshHeader); // 货币变化即刷新顶部资源
    RefreshHeader();
    RefreshEnergyAdButton();
}
public override void OnHide()
{
    eventMgr?.Remove(YOTOEventType.RefreshCurrency, RefreshHeader);
}
public override void OnResize()
{
}
}
// ----- 顶部资源 -----
/// <summary>刷新体力 / 金币显示:胶囊已用图标表示币种(预制体烘焙),这里只填数值,不再写「体力/金币」文字。</summary>
private void RefreshHeader()
{
    if (energyText != null)
        energyText.text = currency != null ? currency.Get(CurrencyType.Energy).ToString() : "0";
    if (goldText != null)
        goldText.text = currency != null ? currency.Get(CurrencyType.Gold).ToString() : "0";
}
// ----- 按钮 -----
// 准备(中间主按钮):先打开选择关卡界面,选好关卡后再进装备界面,最后「出发」进游戏。
private void OnPrepareClick()
{
    Show<MapSelectPanel>();
}
private void OnSettingClick()
{
    Show<SettingPanel>();
}
}

```

```

private void OnShopClick()
{
    Show<ShopPanel>();
}
private void OnCodexClick()
{
    Show<CodexPanel>();
}
private void OnTaskClick()
{
    Show<TaskPanel>();
}
// 排行榜:打开排行榜界面(界面内优先调起 TapTap 内置榜单,不可用则自绘列表)。
private void OnLeaderboardClick()
{
    Show<LeaderboardPanel>();
}
/// <summary>
/// 体力「广告补充」按钮:预留激励广告接口。已注册 <see cref="IAdService"/> 时请求播放,看完回调里补体力并写
盘:
/// 未接入广告(接口未注册)时只记日志、不发奖,避免免费刷体力。
/// </summary>
private void OnEnergyAdClick()
{
    // 每天最多补 DailyAdEnergySystem.DailyLimit 次(次数随存档槽落盘 → 本地+云)
    if (dailyAd != null && !dailyAd.CanWatch)
    {
        GetService<FlyTextMgr>()?.AddTextAtScreenCenter($"今日体力补充已达上限({dailyAd.DailyLimit}次)");
        RefreshEnergyAdButton();
        return;
    }
    if (Context.TryGet<IAdService>(out var ad))
    {
        ad.ShowRewardedAd(AdPlacementEnergy, rewarded =>
        {
            if (!rewarded || currency == null) return;
            currency.Add(CurrencyType.Energy, EnergyAdReward); // 补 1 点体力
            currency.Save(); // 关键节点主动写盘,保证补的体力落地
            dailyAd?.Record(); // 记一次今日已用(只有真看完才计),内部立即落盘
            Context.TryGet<TaskProgressSystem>(out var tp); tp?.AddAdWatch(1); // 计入"观看广告"类任务
            // 通用奖励领取弹窗(Top 层,1 秒自动消失):体力已入账,这里仅展示
            Show<RewardClaimPanel, RewardClaimParam>(new RewardClaimParam
            {
                title = "体力补充",
                rewards = { RewardEntry.Currency(CurrencyType.Energy, EnergyAdReward) },
            });
            RefreshHeader();
            RefreshEnergyAdButton();
        });
    }
}

```

```

else
{
    Debug.Log("[StartPanel] 广告接口(IAdService)未接入,体力广告按钮暂不发奖。接入广告 SDK 后注册 IAdService 即可生效。");
}
}

/// <summary>达上限时把广告补体力按钮置灰不可点。</summary>
private void RefreshEnergyAdButton()
{
    if (btn_energyAd != null) btn_energyAd.interactable = dailyAd == null || dailyAd.CanWatch;
}
}

using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.UI;
using YFramework.Config;
using YOTO;
/// <summary>
/// 任务界面(<see cref="UIEnum.TaskPanel"/>):从主界面「任务」进入。
/// 顶部:返回(关闭) / 资源金币
/// 页签:每日任务 / 常规任务(绿=当前分类)
/// 中部:可滚动任务卡列表,每张卡 = 标题 + 进度(当前/目标) + 奖励(物品/金币/体力 × 数量) + 前往/领取/已领取 按钮
/// 任务定义走配表(<see cref="ConfigManager.taskConfig"/>:task.xlsx → Task.bytes),按 category 分每日/常规两类;
/// 进度与领取状态来自 <see cref="TaskProgressSystem"/>(登录/连登/击杀/看广告/获武器按 objType 自动累计),完成可领,领取发奖并飘字。
/// 进度/领取变化由 <see cref="YOTOEventType.RefreshTask"/> 驱动列表重建。预制体外壳由 <c>Tools/UI/Build TaskPanel Prefab</c> 生成。
/// </summary>
public class TaskPanel : UIPageBase
{
    // 配表 category 取值:与 task.xlsx 第 3 列约定一致
    private const uint CategoryDaily = 1; // 每日任务
    private const uint CategoryRegular = 2; // 常规任务
    [Header("顶部")]
    public Button backBtn;
    public TextMeshProUGUI coinText;
    [Header("页签")]
    public Button tabDaily;
    public Button tabRegular;
    [Header("列表(ScrollRect 的 VerticalLayoutGroup 容器)")]
    public RectTransform content;
    private static readonly Color TabOn = new Color(0.56f, 0.78f, 0.30f, 1f); // 绿:当前分类
    private static readonly Color TabOff = new Color(0.72f, 0.70f, 0.80f, 1f); // 灰紫:未选
    private static readonly Color CardFrame = new Color(0.90f, 0.89f, 0.93f, 1f); // 卡片底框
    private static readonly Color IconBox = new Color(0.78f, 0.80f, 0.86f, 1f); // 物品图标缺失时的占位底
    private static readonly Color CoinIcon = new Color(0.45f, 0.74f, 0.36f, 1f); // 金币图标(绿色块占位)
    private static readonly Color EnergyIcon = new Color(0.36f, 0.62f, 0.86f, 1f); // 体力图标(蓝色块占位)
    private static readonly Color GotoBtn = new Color(0.96f, 0.66f, 0.18f, 1f); // 「前往」橙钮

```

```

private static readonly Color ClaimBtn = new Color(0.30f, 0.74f, 0.36f, 1f); // 「领取」绿钮(已完成可领)
private static readonly Color ClaimedBtn = new Color(0.62f, 0.62f, 0.66f, 1f); // 「已领取」灰钮(不可点)
private static readonly Color TitleText = new Color(0.25f, 0.24f, 0.30f, 1f);
private ConfigManager config;
private CurrencySystem currency;
private ResMgr resMgr;
private EventMgr eventMgr;
private TaskProgressSystem taskProgress;
private TMP_FontAsset font;
private readonly List<Task> entries = new List<Task>();
private uint currentCategory = CategoryDaily;
public override void OnLoad()
{
    config = GetService<ConfigManager>();
    currency = GetService<CurrencySystem>();
    resMgr = GetService<ResMgr>();
    eventMgr = GetService<EventMgr>();
    taskProgress = GetService<TaskProgressSystem>();
    if (coinText != null) font = coinText.font; // 复用外壳字体给运行时卡片
    if (backBtn != null) backBtn.onClick.AddListener(CloseSelf);
    if (tabDaily != null) tabDaily.onClick.AddListener(() => SelectCategory(CategoryDaily));
    if (tabRegular != null) tabRegular.onClick.AddListener(() => SelectCategory(CategoryRegular));
}
public override void OnShow()
{
    eventMgr?.Add(YOTOEventType.RefreshCurrency, RefreshCoin);
    eventMgr?.Add(YOTOEventType.RefreshTask, RebuildList); // 任务进度/领取变化即重建当前列表
    SelectCategory(currentCategory);
    RefreshCoin();
}
public override void OnHide()
{
    eventMgr?.Remove(YOTOEventType.RefreshCurrency, RefreshCoin);
    eventMgr?.Remove(YOTOEventType.RefreshTask, RebuildList);
}
public override void OnResize() { }
// ----- 顶部 -----
private void RefreshCoin()
{
    if (coinText == null || currency == null) return;
    coinText.text = currency.Get(CurrencyType.Gold).ToString();
}
// ----- 页签 -----
private void SelectCategory(uint category)
{
    currentCategory = category;
    SetTabColor(tabDaily, category == CategoryDaily);
    SetTabColor(tabRegular, category == CategoryRegular);
    ReloadEntries();
    RebuildList();
}

```

```

}
private static void SetTabColor(Button btn, bool on)
{
    if (btn == null) return;
    if (btn.targetGraphic is Image img) img.color = on ? TabOn : TabOff;
}
/// <summary>从配表拉取当前分类任务,按 SortPriority 升序。</summary>
private void ReloadEntries()
{
    entries.Clear();
    if (config == null) return;
    foreach (var kv in config.taskConfig.items)
        if (kv.Value.Category == currentCategory) entries.Add(kv.Value);
    entries.Sort((a, b) => a.SortPriority.CompareTo(b.SortPriority));
}
/// <summary>当前进度:取自 <see cref="TaskProgressSystem"/>(登录/连登/击杀/看广告/获武器按 objType 自动累计)。</summary>
private int CurrentProgress(uint taskId) => taskProgress != null ? taskProgress.GetProgress(taskId) : 0;
// ----- 列表 -----
private void RebuildList()
{
    if (content == null) return;
    for (int i = content.childCount - 1; i >= 0; i--) Destroy(content.GetChild(i).gameObject);
    foreach (var task in entries) BuildCard(task);
}
private void BuildCard(Task task)
{
    // 卡片底框(高度固定,宽度由 VerticalLayoutGroup 撑满)
    var card = NewChild(content, $"Task_{task.Id}", out var cardRt);
    var bg = card.AddComponent<Image>();
    bg.color = CardFrame;
    var le = card.AddComponent<LayoutElement>();
    le.preferredHeight = 340; // 字体 ×2 后加高卡片,容下标题/进度/奖励/按钮
    // 标题(左上,带高 104 容下 46pt×2)
    var title = NewChild(card.transform, "Title", out var titleRt);
    titleRt.anchorMin = new Vector2(0, 1); titleRt.anchorMax = new Vector2(0, 1); titleRt.pivot = new Vector2(0, 1);
    titleRt.anchoredPosition = new Vector2(40, -30); titleRt.sizeDelta = new Vector2(760, 104);
    NewText(title, task.Name, 46, TitleText, TextAlignmentOptions.Left);
    // 进度(右上,带高 104 容下字号;加宽并关掉省略,避免「/ 目标」被裁成 ...):进度 cur/need
    var prog = NewChild(card.transform, "Progress", out var progRt);
    progRt.anchorMin = new Vector2(1, 1); progRt.anchorMax = new Vector2(1, 1); progRt.pivot = new Vector2(1, 1);
    progRt.anchoredPosition = new Vector2(-30, -30); progRt.sizeDelta = new Vector2(520, 104);
    var progText = NewText(prog, "", 38, TitleText, TextAlignmentOptions.Right);
    progText.overflowMode = TextOverflowModes.Overflow; // 不省略,完整显示「进度 x / y」
    progText.text = $"进度 <color=#D94C40>{CurrentProgress(task.Id)}</color> / {task.TargetAmount}";
    // 奖励(左下):有物品才显示物品×数量,再金币×数量,再体力×数量(色块占位)
    float x = 40;
    if (task.RewardItemId > 0 && task.RewardItemCount > 0)

```



```

        x = BuildReward(card.transform, ItemIcon(task.RewardItemId), IconBox, task.RewardItemCount, x);
    if (task.RewardCoin > 0)
        x = BuildReward(card.transform, null, CoinIcon, task.RewardCoin, x);
    if (task.RewardEnergy > 0)
        x = BuildReward(card.transform, null, EnergyIcon, task.RewardEnergy, x);
    // 右下按钮:未完成=前往(关闭去做);已完成未领=领取(发奖);已领取=置灰不可点
    bool canClaim = taskProgress != null && taskProgress.CanClaim(task.Id);
    bool claimed = taskProgress != null && taskProgress.IsClaimed(task.Id);
    var go = NewChild(card.transform, "Action", out var gotoRt);
    gotoRt.anchorMin = new Vector2(1, 0); gotoRt.anchorMax = new Vector2(1, 0); gotoRt.pivot = new Vector2(
1, 0);
    gotoRt.anchoredPosition = new Vector2(-40, 44); gotoRt.sizeDelta = new Vector2(240, 120);
    var gotoImg = go.AddComponent<Image>();
    gotoImg.color = claimed ? ClaimedBtn : (canClaim ? ClaimBtn : GotoBtn);
    var gotoBtn = go.AddComponent<Button>();
    gotoBtn.targetGraphic = gotoImg;
    gotoBtn.interactable = !claimed;
    gotoBtn.onClick.AddListener(() => OnActionClick(task));
    var gotoLabel = NewChild(go.transform, "Label", out var gotoLabelRt);
    Stretch(gotoLabelRt);
    NewText(gotoLabel, claimed ? "已领取" : (canClaim ? "领取" : "前往"), 40, Color.white, TextAlignmentOpt
ions.Center);
    }
    /// <summary>一格奖励:图标(无 sprite 时显示色块) + ×数量,返回下一格的起始 x。</summary>
    private float BuildReward(Transform parent, Sprite sprite, Color iconColor, int count, float x)
    {
        const float iconSize = 88f;
        var icon = NewChild(parent, "RewardIcon", out var iconRt);
        iconRt.anchorMin = new Vector2(0, 0); iconRt.anchorMax = new Vector2(0, 0); iconRt.pivot = new Vector2(
0, 0);
        iconRt.anchoredPosition = new Vector2(x, 48); iconRt.sizeDelta = new Vector2(iconSize, iconSize);
        var img = icon.AddComponent<Image>();
        img.preserveAspect = true;
        img.raycastTarget = false;
        if (sprite != null) { img.sprite = sprite; img.color = Color.white; }
        else img.color = iconColor; // 缺图标时用色块占位
        var cnt = NewChild(parent, "RewardCount", out var cntRt);
        cntRt.anchorMin = new Vector2(0, 0); cntRt.anchorMax = new Vector2(0, 0); cntRt.pivot = new Vector2(0,
0);
        cntRt.anchoredPosition = new Vector2(x + iconSize + 8, 48); cntRt.sizeDelta = new Vector2(240, iconSize
);
        NewText(cnt, $"×{count}", 36, TitleText, TextAlignmentOptions.Left);
        return x + iconSize + 280; // 图标 + 数量(×2 字号加宽) + 间距
    }
    /// <summary>取奖励物品的图标(配表 iconPath → Resources 精灵);取不到返回 null,由调用方用色块占位。</summary>
    >
    private Sprite ItemIcon(uint itemId)
    {
        if (itemId == 0 || config == null || resMgr == null) return null;
        var item = config.itemConfig.Get(itemId);

```

```

        if (item == null || string.IsNullOrEmpty(item.IconPath)) return null;
        return resMgr.Load<Sprite>(item.IconPath);
    }
    /// <summary>右下按钮点击:可领取一领奖并飘字(列表由 RefreshTask 事件自动重建);否则前往(关闭界面去做)。</summary>
    private void OnActionClick(Task task)
    {
        if (taskProgress != null && taskProgress.CanClaim(task.Id))
        {
            if (taskProgress.Claim(task.Id))
            {
                ShowRewardPopup(task); // 奖励已由 Claim 入账,这里弹通用领取弹窗展示
                return; // 领取后 RefreshTask 会重建列表,无需手动刷新
            }
            if (taskProgress != null && taskProgress.IsClaimed(task.Id)) return; // 已领取:无操作
            CloseSelf(); // 未完成:前往(回大厅去做任务)。后续可按任务类型路由到具体玩法。
        }
        /// <summary>领取成功后弹通用奖励领取弹窗(Top 层,1 秒自动消失):按任务配表的物品/金币/体力组奖励列表。</summary>
        private void ShowRewardPopup(Task task)
        {
            var rewards = new List<RewardEntry>();
            if (task.RewardItemId > 0 && task.RewardItemCount > 0)
            {
                rewards.Add(RewardEntry.Item((int)task.RewardItemId, task.RewardItemCount));
            }
            if (task.RewardCoin > 0) rewards.Add(RewardEntry.Currency(CurrencyType.Gold, task.RewardCoin));
            if (task.RewardEnergy > 0) rewards.Add(RewardEntry.Currency(CurrencyType.Energy, task.RewardEnergy));
            if (rewards.Count == 0) return;
            Show<RewardClaimPanel, RewardClaimParam>(new RewardClaimParam { title = "任务奖励", rewards = rewards });
        }
        // ----- 工具 -----
        private GameObject NewChild(Transform parent, string name, out RectTransform rt)
        {
            var go = new GameObject(name, typeof(RectTransform));
            rt = (RectTransform)go.transform;
            rt.SetParent(parent, false);
            return go;
        }
        private static void Stretch(RectTransform rt)
        {
            rt.anchorMin = Vector2.zero; rt.anchorMax = Vector2.one;
            rt.offsetMin = Vector2.zero; rt.offsetMax = Vector2.zero;
        }
        private TextMeshProUGUI NewText(GameObject go, string text, float size, Color color, TextAlignmentOptions align)
        {
            var tmp = go.AddComponent<TextMeshProUGUI>();
            tmp.text = text; tmp.fontSize = UITheme.Font(size); tmp.alignment = align; tmp.color = color;
            tmp.raycastTarget = false; tmp.enableWordWrapping = false; tmp.overflowMode = TextOverflowModes.Ellipsis;
            var f = font != null ? font : TMP_Settings.defaultFontAsset;

```

```

        if (f != null) tmp.font = f;
        return tmp;
    }
}
using UnityEngine;
/// <summary>挂在物体上让其绕指定轴匀速自转(UI 模型预览的转台用)。</summary>
public class AutoRotate : MonoBehaviour
{
    public float degPerSecond = 30f;
    public Vector3 axis = Vector3.up;
    private void Update()
    {
        transform.Rotate(axis, degPerSecond * Time.deltaTime, Space.Self);
    }
}
using UnityEngine;
using YFramework.Config;
using YOTO;
/// <summary>
/// 第一人称手持武器视图模型:挂在主相机上,把当前出战武器(<see cref="LoadoutSystem"/> 选中的枪)的模型
/// 作为相机子物体放在镜头前方,像 FPS 那样"端在手里"。换装(<see cref="YOTOEventType.RefreshLoadout"/>)时自动
/// 更换。
///
/// 由 <see cref="GameStartScene"/> 进对局时 AddComponent 并 <see cref="Init"/> 注入 <see cref="GameContext"/>
/// 。
/// 手持的局部位置/旋转/缩放是公开字段,觉得位置不对直接调这几个常量即可。
/// </summary>
public class FpsWeaponViewModel : MonoBehaviour
{
    [Header("手持视角:相对相机的局部变换(觉得不对就调这几个)")]
    public Vector3 localPosition = new Vector3(0.22f, -0.29f, 0.42f); // 右、下、前(编辑器内实测值)
    public Vector3 localEuler = new Vector3(-5f, 77.48f, -12.57f); // 枪口朝向(编辑器内实测值)
    public float scale = 1f;
    [Header("开枪后座(代码模拟:冲击后退上抬 + 平滑复位)")]
    public float kickBack = 0.06f; // 每枪向后位移(米)
    public float kickUp = 6f; // 每枪枪口上抬(度)
    public float kickRandomYaw = 2.5f; // 每枪左右随机偏摆(度)
    public float snappiness = 18f; // 后座到位速度(越大越脆)
    public float returnSpeed = 9f; // 复位速度(越大回得越快)
    private LoadoutSystem loadout;
    private ConfigManager config;
    private ResMgr res;
    private EventMgr eventMgr;
    private ScopeAimController scopeAim; // 同挂主相机:开镜时据此隐藏手持枪
    private GameObject model;
    private int shownId = -1;
    private bool forceHidden; // 结束打猎尸检镜头期间强制隐藏手持枪
    /// <summary>强制隐藏/显示手持枪(尸检镜头用;进对局 <see cref="Init"/> 会复位为显示)。</summary>
    public void SetForceHidden(bool hidden) => forceHidden = hidden;
    private Vector3 recoilPos, recoilEuler; // 当前后座偏移(叠加在 rest 之上)

```

```

private Vector3 targetRecoilPos, targetRecoilEuler; // 目标后座(开枪冲击后向 0 衰减)
/// <summary>注入服务并开始随换装刷新。GameStartScene 在挂载后调用。</summary>
public void Init(GameContext ctx)
{
    forceHidden = false; // 每次进对局复位:正常显示手持枪
    loadout = ctx.Get<LoadoutSystem>();
    config = ctx.Get<ConfigManager>();
    res = ctx.Get<ResMgr>();
    eventMgr = ctx.Get<EventMgr>();
    scopeAim = GetComponent<ScopeAimController>();
    eventMgr?.Add(YOTOEventType.RefreshLoadout, Refresh);
    eventMgr?.Add(YOTOEventType.Shoot, OnShoot); // 开枪后座
    Refresh();
}
private void OnDestroy()
{
    eventMgr?.Remove(YOTOEventType.RefreshLoadout, Refresh);
    eventMgr?.Remove(YOTOEventType.Shoot, OnShoot);
    if (model != null) Destroy(model);
}
/// <summary>每帧把后座偏移弹簧式逼近并衰减回 0,叠加到手持 rest 姿势上。</summary>
private void Update()
{
    if (model == null) return;
    // 开镜(瞄准镜)时隐藏手持枪,退出瞄准再显示;尸检镜头期间强制隐藏
    bool show = !forceHidden && (scopeAim == null || !scopeAim.IsAiming);
    if (model.activeSelf != show) model.SetActive(show);
    float dt = Time.deltaTime;
    // 目标后座向 0 衰减(恢复);当前后座向目标快速逼近(冲击)→ 脆击 + 平滑回复
    targetRecoilPos = Vector3.Lerp(targetRecoilPos, Vector3.zero, returnSpeed * dt);
    targetRecoilEuler = Vector3.Lerp(targetRecoilEuler, Vector3.zero, returnSpeed * dt);
    recoilPos = Vector3.Lerp(recoilPos, targetRecoilPos, snappiness * dt);
    recoilEuler = Vector3.Lerp(recoilEuler, targetRecoilEuler, snappiness * dt);
    model.transform.localPosition = localPosition + recoilPos;
    model.transform.localEulerAngles = localEuler + recoilEuler;
}
/// <summary>每次开枪(<see cref="YOTOEventType.Shoot"/>)给一记后座冲击:向后 + 枪口上抬 + 轻微随机偏摆。</summary>
private void OnShoot()
{
    targetRecoilPos += new Vector3(0f, 0f, -kickBack); // 向相机方向(后)退
    targetRecoilEuler += new Vector3(-kickUp, Random.Range(-kickRandomYaw, kickRandomYaw), 0f); // -X=枪口
    // 上抬
}
/// <summary>按当前出战武器的 item.ModelPath 换上手持模型;无模型则不显示。</summary>
private void Refresh()
{
    int id = loadout != null ? loadout.GetSelected(ShopCategory.Weapon) : 0;
    var item = id > 0 ? config?.itemConfig.Get((uint)id) : null;
    string path = item != null ? item.ModelPath : null;

```

```

if (string.IsNullOrEmpty(path))
{
    if (model != null) { Destroy(model); model = null; }
    shownId = -1;
    return;
}
if (id == shownId && model != null) return; // 没变
if (model != null) { Destroy(model); model = null; }
var prefab = res != null ? res.Load<GameObject>(path) : null;
if (prefab == null)
{
    Debug.LogWarning($"[FpsWeaponViewModel] 武器模型未找到(确认已在 Resources/ 下): {path}");
    return;
}
Debug.Log($"[FpsWeaponViewModel] 出战武器 id={id} 手持模型={path}");
model = Instantiate(prefab, transform); // 作为相机子物体
WeaponModelUtil.DisableColliders(model); // 去碰撞体,避免挡住自己的射线/物理
var t = model.transform;
t.localPosition = localPosition;
t.localEulerAngles = localEuler;
t.localScale = Vector3.one * scale;
recoilPos = recoilEuler = targetRecoilPos = targetRecoilEuler = Vector3.zero; // 换枪清掉残余后座
shownId = id;
}
}

using UnityEngine;
using UnityEngine.Rendering;
using UnityEngine.Rendering.Universal;
using YOTO;

/// <summary>
/// 一次性「模型快照」:用一台共用的离屏相机把 Resources 里的模型渲染**一帧**到 Texture2D 返回,然后相机闲置(不常驻渲染)。
/// 适合需要很多静态小图的场景(如装备选择列表):比每张卡挂一台常驻相机省得多。
/// URP 用 <see cref="RenderPipeline.SubmitRenderRequest"/> 同步渲染单帧(Unity 2022.2+);失败回退 Camera.Render。
///
/// 注意:同步 ReadPixels 会有一次性 GPU→CPU 等待,批量生成时集中在打开界面那一帧,属一次性开销。
/// 返回的 Texture2D 由调用方负责 Destroy 回收。
/// </summary>
public static class ModelSnapshot
{
    private const int PreviewLayer = 18; // 复用 WeaponPreview 层
    private static readonly Vector3 RigOrigin = new Vector3(8000f, 8000f, 0f); // 远离场景与其它预览 rig
    private static GameObject rig;
    private static Transform pivot;
    private static Camera cam;
    /// <summary>把模型渲一帧成 Texture2D(侧视、不旋转)。失败返回 null。<paramref name="bg"/> 用透明可衬卡片底色。</summary>
    public static Texture2D Capture(string modelPath, ResMgr res, int size = 256, bool sideView = true, Color bg = default)

```

```

{
    if (string.IsNullOrEmpty(modelPath) || res == null) return null;
    var prefab = res.Load<GameObject>(modelPath);
    if (prefab == null)
    {
        Debug.LogWarning($"[ModelSnapshot] 模型未找到: {modelPath}");
        return null;
    }
    EnsureRig();
    var model = Object.Instantiate(prefab, pivot);
    // 弱点高亮/命中盒不入镜
    foreach (var hz in model.GetComponentsInChildren<AnimalHitZone>(true)) hz.gameObject.SetActive(false);
    WeaponModelUtil.SetLayer(model, PreviewLayer);
    WeaponModelUtil.DisableColliders(model);
    WeaponModelUtil.MakeUnlit(model);
    WeaponModelUtil.Frame(cam, pivot, model); // 居中 + 默认取景
    if (sideView) FrameSide(model); // 改正侧视图
    var rt = RenderTexture.GetTemporary(size, size, 16, RenderTextureFormat.ARGB32);
    cam.targetTexture = rt;
    cam.backgroundColor = bg;
    // —— 同步渲一帧 ——
    var req = new UniversalRenderPipeline.SingleCameraRequest { destination = rt };
    if (RenderPipeline.SupportsRenderRequest(cam, req)) RenderPipeline.SubmitRenderRequest(cam, req);
    else cam.Render(); // 非 URP 回退
    // —— 读回像素 ——
    var prevActive = RenderTexture.active;
    RenderTexture.active = rt;
    var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
    tex.ReadPixels(new Rect(0, 0, size, size), 0, 0);
    tex.Apply();
    RenderTexture.active = prevActive;
    cam.targetTexture = null;
    RenderTexture.ReleaseTemporary(rt);
    // 清理:MakeUnlit 新建的材质 + 模型实例,避免泄漏(同步销毁,以免下一次快照重叠入镜)
    foreach (var r in model.GetComponentsInChildren<Renderer>(true))
        foreach (var m in r.sharedMaterials)
            if (m != null) Object.DestroyImmediate(m);
    Object.DestroyImmediate(model);
    return tex;
}

private static void EnsureRig()
{
    if (rig != null && cam != null) return; // 注意:场景切换销毁后 Unity 假 null,会重建
    rig = new GameObject("ModelSnapshotRig") { hideFlags = HideFlags.HideAndDontSave };
    rig.transform.position = RigOrigin;
    var pv = new GameObject("Pivot");
    pv.transform.SetParent(rig.transform, false);
    pivot = pv.transform;
    var camGo = new GameObject("SnapshotCam");
    camGo.transform.SetParent(rig.transform, false);
}

```

```

cam = camGo.AddComponent<Camera>();
cam.enabled = false; // 不常驻渲染,只在 SubmitRenderRequest 时渲一帧
cam.clearFlags = CameraClearFlags.SolidColor;
cam.cullingMask = 1 << PreviewLayer;
cam.fieldOfView = 30f;
cam.nearClipPlane = 0.01f;
cam.farClipPlane = 50f;
}
/// <summary>把相机摆到模型正侧面(垂直于较长水平轴),略抬高。</summary>
private static void FrameSide(GameObject model)
{
    var rs = model.GetComponentsInChildren<Renderer>();
    if (rs.Length == 0) return;
    var b = rs[0].bounds;
    for (int i = 1; i < rs.Length; i++) b.Encapsulate(rs[i].bounds);
    float radius = Mathf.Max(b.extents.magnitude, 0.05f);
    float dist = radius / Mathf.Sin(Mathf.Deg2Rad * cam.fieldOfView * 0.5f) * 1.15f;
    Vector3 dir = (b.size.z >= b.size.x) ? new Vector3(1f, 0.12f, 0f) : new Vector3(0f, 0.12f, 1f);
    dir = dir.normalized;
    cam.transform.position = b.center + dir * dist;
    cam.transform.LookAt(b.center);
}
}
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using YOTO;
/// <summary>
/// 道具卡片快照的全局共享缓存。把 <see cref="ModelSnapshot.Capture"/> 渲染出的 3D 道具侧视快照按 modelPath 缓存,
/// 装备页 / 商城页 / 图鉴等卡片列表共用同一份——**同一把武器全局只渲一次**,跨面板、跨多次打开都复用。
///
/// **统一规格**(256 侧视 + 卡片底色),保证各页面卡片图视觉一致。卡片显示用 <see cref="CardSprite"/>:
/// 把快照包成 Sprite,配 <c>Image.preserveAspect=true</c> 即可按比例居中,不会被卡片图框拉伸(RawImage 无 preserveAspect)。
/// **生命周期**:进程级常驻(不随面板关闭释放),换取"渲过即复用";道具种类有限(数十),内存可控(~256KB/张)。
/// 需要主动释放可调 <see cref="Clear"/>(注意:释放后仍在显示这些图的 Image 会变空白)。
/// </summary>
public static class ModelSnapshotCache
{
    private static readonly Color CardBg = new Color(0.12f, 0.13f, 0.16f, 1f); // 与原装备页 SnapshotBg 一致
    private static readonly Dictionary<string, Texture2D> texCache = new();
    private static readonly Dictionary<string, Sprite> spriteCache = new();
    /// <summary>取道具卡片侧视快照贴图(命中即复用;未命中渲一次并缓存,失败的 null 也缓存避免反复重试)。</summary>
y>
    public static Texture2D CardSnapshot(string modelPath, ResMgr res)
    {
        if (string.IsNullOrEmpty(modelPath) || res == null) return null;
        if (texCache.TryGetValue(modelPath, out var tex)) return tex; // 含失败缓存的 null
        tex = ModelSnapshot.Capture(modelPath, res, 256, sideView: true, bg: CardBg);
        texCache[modelPath] = tex;
    }
}

```

```

        return tex;
    }
    /// <summary>取道具卡片快照的 Sprite(包住缓存贴图,本身也缓存)。配 Image.preserveAspect 显示,避免拉伸。无模
    型回 null。</summary>
    public static Sprite CardSprite(string modelPath, ResMgr res)
    {
        if (string.IsNullOrEmpty(modelPath) || res == null) return null;
        if (spriteCache.TryGetValue(modelPath, out var sp)) return sp;
        var tex = CardSnapshot(modelPath, res);
        sp = tex != null ? Sprite.Create(tex, new Rect(0, 0, tex.width, tex.height), new Vector2(0.5f, 0.5f), 1
00f) : null;
        spriteCache[modelPath] = sp;
        return sp;
    }
    /// <summary>释放所有已缓存的快照 Sprite 与贴图并清空(下次取再重渲)。</summary>
    public static void Clear()
    {
        foreach (var s in spriteCache.Values) if (s != null) Object.Destroy(s);
        foreach (var t in texCache.Values) if (t != null) Object.Destroy(t);
        spriteCache.Clear();
        texCache.Clear();
    }
}

using UnityEngine;
using UnityEngine.UI;
using YOTO;
/// <summary>
/// UI 里的武器 3D 模型预览(转台)。用一台只拍 <see cref="PreviewLayer"/> 层的离屏相机渲染到 RenderTexture,
/// 贴到调用方给的 <see cref="RawImage"/> 宿主上;模型在远离场景处缓慢自转,换成 Unlit 自发光以免受场景灯光影响。
///
/// 用法(面板里):
/// var preview = new WeaponModelPreview(hostRect, resMgr);
/// preview.Show("Weapon/AssaultRifle1_01"); // 切换展示的模型(空字符串=清空)
/// preview.SetActive(false);           // 面板隐藏时停渲染省开销
/// preview.Dispose();                   // 彻底回收
///
/// 商店与装备界面共用此类(见 <see cref="ShopPanel"/> / <see cref="EquipPanel"/>)。
/// </summary>
public class WeaponModelPreview
{
    private const int PreviewLayer = 18; // ProjectSettings 中的 "WeaponPreview" 层
    private static int s_count;          // 给每个实例一个独立的世界原点,避免多预览同层互相入镜
    private readonly ResMgr res;
    private readonly RawImage image;
    private readonly Vector3 origin;
    private readonly bool spin;          // 是否自转(转台);false=固定不转
    private readonly bool sideView;      // 是否用正侧视图(否则默认 3/4 取景)
    private readonly int rtSize;         // RenderTexture 边长
    private RenderTexture rt;
    private Camera cam;

```



```

private GameObject rig;
private Transform pivot;
private GameObject model;
private string shownPath;
/// <param name="spin">是否缓慢自转(转台);装备卡用 false=不转。</param>
/// <param name="sideView">true=正侧视图(看模型侧面);false=默认 3/4 取景。</param>
/// <param name="rtSize">离屏贴图分辨率;卡片小图可调小省开销。</param>
public WeaponModelPreview(RectTransform host, ResMgr res, bool spin = true, bool sideView = false, int rtSize = 640)
{
    this.res = res;
    this.spin = spin;
    this.sideView = sideView;
    this.rtSize = Mathf.Max(64, rtSize);
    origin = new Vector3(s_count++ * 1000f, 5000f, 0f); // 间距 1000 远大于相机远裁剪 50,彼此看不到
    image = host.GetComponent<RawImage>();
    if (image == null) image = host.gameObject.AddComponent<RawImage>();
    image.raycastTarget = false;
    image.enabled = false;
}
/// <summary>展示指定 Resources 路径的武器模型(无扩展名)。传空则清空隐藏。</summary>
public void Show(string modelPath)
{
    if (string.IsNullOrEmpty(modelPath)) { Clear(); return; }
    EnsureRig();
    if (modelPath == shownPath && model != null) { image.enabled = true; return; }
    ClearModel();
    var prefab = res != null ? res.Load<GameObject>(modelPath) : null;
    if (prefab == null)
    {
        Debug.LogWarning($"[WeaponModelPreview] 模型未找到(确认已在 Resources/ 下): {modelPath}");
        image.enabled = false;
        return;
    }
    model = Object.Instantiate(prefab, pivot);
    // 动物预制体带弱点高亮/命中盒(AnimalHitZone),UI 预览里不该显示:整块关掉
    foreach (var hz in model.GetComponentsInChildren<AnimalHitZone>(true)) hz.gameObject.SetActive(false);
    WeaponModelUtil.SetLayer(model, PreviewLayer);
    WeaponModelUtil.DisableColliders(model);
    WeaponModelUtil.MakeUnlit(model);
    WeaponModelUtil.Frame(cam, pivot, model); // 先居中 + 默认 3/4 取景
    if (sideView) SideFrame(); // 需要则改为正侧视图
    shownPath = modelPath;
    image.enabled = true;
}
/// <summary>把相机摆到模型正侧面(垂直于较长水平轴看其侧面),略微抬高。</summary>
private void SideFrame()
{
    var rs = model.GetComponentsInChildren<Renderer>();
    if (rs.Length == 0) return;

```

```

var b = rs[0].bounds;
for (int i = 1; i < rs.Length; i++) b.Encapsulate(rs[i].bounds);
float radius = Mathf.Max(b.extents.magnitude, 0.05f);
float dist = radius / Mathf.Sin(Mathf.Deg2Rad * cam.fieldOfView * 0.5f) * 1.15f;
Vector3 dir = (b.size.z >= b.size.x) ? new Vector3(1f, 0.12f, 0f) : new Vector3(0f, 0.12f, 1f);
dir = dir.normalized;
cam.transform.position = b.center + dir * dist;
cam.transform.LookAt(b.center);
}
/// <summary>显隐:整体启用/停用预览 rig(而非只关相机)。
/// 注意必须切 rig 的 active 而不是 cam.enabled——后者被禁用再启用后 RenderTexture 不会重新渲染,
/// 导致切走页签再回来时模型不再出现(只剩卡片描边)。切 GameObject 的 active 则会强制重渲一帧。</summary>
public void SetActive(bool on)
{
    if (rig != null && rig.activeSelf != on) rig.SetActive(on);
    if (image != null) image.enabled = on && model != null;
}
public void Clear()
{
    ClearModel();
    shownPath = null;
    if (image != null) image.enabled = false;
}
public void Dispose()
{
    ClearModel();
    if (rig != null) { Object.Destroy(rig); rig = null; cam = null; pivot = null; }
    if (rt != null) { rt.Release(); Object.Destroy(rt); rt = null; }
    if (image != null) image.enabled = false;
    shownPath = null;
}
private void ClearModel()
{
    if (model != null) { Object.Destroy(model); model = null; }
}
/// <summary>懒建(并在被场景卸载销毁后重建)离屏相机 + 转台 + RenderTexture。</summary>
private void EnsureRig()
{
    if (rt == null)
    {
        rt = new RenderTexture(rtSize, rtSize, 16, RenderTextureFormat.ARGB32) { name = "WeaponPreviewRT" };
        rt.Create();
    }
    if (image != null) image.texture = rt;
    if (rig != null) return;
    // rig 被场景切换销毁后,其子物体(pivot/model)也没了:重置 shownPath 以便重新加载
    shownPath = null;
    model = null;
    rig = new GameObject("WeaponPreviewRig");

```

```

    rig.transform.position = origin;
    var pv = new GameObject("Pivot");
    pv.transform.SetParent(rig.transform, false);
    pivot = pv.transform;
    if (spin) pv.AddComponent<AutoRotate>(); // 不转的卡片不挂自转
    var camGo = new GameObject("PreviewCam");
    camGo.transform.SetParent(rig.transform, false);
    cam = camGo.AddComponent<Camera>();
    cam.clearFlags = CameraClearFlags.SolidColor;
    // 侧视卡片用透明底(衬出卡片色);默认大预览用深色底板
    cam.backgroundColor = sideView ? new Color(0f, 0f, 0f, 0f) : new Color(0.10f, 0.11f, 0.13f, 0.85f);
    cam.cullingMask = 1 << PreviewLayer;           // 只拍武器
    cam.fieldOfView = 30f;
    cam.nearClipPlane = 0.01f;
    cam.farClipPlane = 50f;
    cam.targetTexture = rt;
}
}
using UnityEngine;
/// <summary>
/// 武器模型展示的公用处理:设层 / 换 Unlit 自发光材质 / 去碰撞体 / 按包围盒取景。
/// 被 <see cref="WeaponModelPreview"/>(UI 离屏预览)与 <see cref="FpsWeaponViewModel"/>(第一人称手持)共用。
/// </summary>
public static class WeaponModelUtil
{
    /// <summary>递归设置层。</summary>
    public static void SetLayer(GameObject go, int layer)
    {
        go.layer = layer;
        var t = go.transform;
        for (int i = 0; i < t.childCount; i++) SetLayer(t.GetChild(i).gameObject, layer);
    }
    /// <summary>关掉模型上的所有碰撞体(LowPolyWeapons 预制体自带 MeshCollider,展示/手持时不该参与物理或射线)。</summary>
    public static void DisableColliders(GameObject go)
    {
        foreach (var c in go.GetComponentsInChildren<Collider>(true)) c.enabled = false;
    }
    /// <summary>Resources 下的 URP/Unlit 模板材质路径(去后缀)。</summary>
    private const string UnlitTemplatePath = "Materials/UnlitPreview";
    private static Material unlitTemplate;
    /// <summary>
    /// 把材质换成 Unlit(保留主纹理/颜色),使其不依赖任何灯光即可清晰显示(UI 离屏预览 / 第一人称手持)。
    ///
    /// **真机务必走模板材质**:运行时 <c>new Material(Shader.Find("URP/Unlit"))</c> 没有任何序列化资产引用该 sh
    ader 变体,
    /// 打包时会被 URP 的变体裁剪删掉 → 真机渲染成品红(editor 因为全量变体在所以正常)。
    /// 改为从 <c>Resources/Materials/UnlitPreview.mat</c>(序列化资产,变体会被打包器收集)<c>Instantiate</c>,
    /// 只改主纹理/颜色不动 keyword,保持同一变体,真机即可正常显示。模板缺失时退回 Shader.Find(仅作 editor/兜底)
    。

```

```

/// </summary>
public static void MakeUnlit(GameObject go)
{
    if (unlitTemplate == null) unlitTemplate = Resources.Load<Material>(UnlitTemplatePath);
    Shader fallbackShader = null;
    if (unlitTemplate == null)
    {
        fallbackShader = Shader.Find("Universal Render Pipeline/Unlit");
        if (fallbackShader == null) fallbackShader = Shader.Find("Unlit/Texture");
        if (fallbackShader == null) return;
    }
    foreach (var r in go.GetComponentsInChildren<Renderer>(true))
    {
        var srcs = r.sharedMaterials;
        var dsts = new Material[srcs.Length];
        for (int i = 0; i < srcs.Length; i++)
        {
            var m = unlitTemplate != null ? new Material(unlitTemplate) : new Material(fallbackShader);
            var src = srcs[i];
            if (src != null)
            {
                // 读属性前先判存在,否则对缺该属性的 shader(如 Hunting/WeakPointHighlight 无 _MainTex/_Color
                // mainTexture/color 的 getter 会逐个刷 "doesn't have a texture property '_MainTex'" 报错。
                if (src.HasProperty("_MainTex") && src.mainTexture != null) m.mainTexture = src.mainTexture;

                else if (src.HasProperty("_BaseColor")) m.color = src.GetColor("_BaseColor");
                else if (src.HasProperty("_Color")) m.color = src.color;
            }
            dsts[i] = m;
        }
        r.sharedMaterials = dsts;
    }
}

/// <summary>把模型包围盒中心对齐到 pivot,并按其大小自动拉远相机取景(前-右-上 3/4 视角)。</summary>
public static void Frame(Camera cam, Transform pivot, GameObject model)
{
    if (cam == null || pivot == null || model == null) return;
    var renderers = model.GetComponentsInChildren<Renderer>();
    if (renderers.Length == 0) return;
    var b = renderers[0].bounds;
    for (int i = 1; i < renderers.Length; i++) b.Encapsulate(renderers[i].bounds);
    model.transform.position += pivot.position - b.center; // 中心对齐 pivot
    float radius = Mathf.Max(b.extents.magnitude, 0.05f);
    float dist = radius / Mathf.Sin(Mathf.Deg2Rad * cam.fieldOfView * 0.5f) * 1.15f;
    Vector3 dir = new Vector3(0.6f, 0.35f, -1f).normalized;
    cam.transform.position = pivot.position + dir * dist;
    cam.transform.LookAt(pivot.position);
}
}

```